

INFORME FINAL — ERSO 2026-004

- [INFORME FINAL — ERSO 2026-004](#)
 - [Despliegue del registro verificable de datos \(VDR\) de extensión](#)
 - [Cómo leer este informe](#)
 - [Contenido](#)
 - [Resumen ejecutivo](#)
- [PARTE I — MARCO CONCEPTUAL: los términos que necesitas](#)
 - [1. La historia en cinco minutos](#)
 - [2. Diccionario de términos](#)
 - [3. Las siete ideas madre](#)
 - [4. El método de trabajo: cómo se piensa un criterio](#)
- [PARTE II — QUÉ PIDE LA ERSO 004, EXPLICADO FRASE POR FRASE](#)
 - [5. La ficha del documento](#)
 - [6. Capacidades del proceso](#)
 - [7. Condiciones del proceso](#)
 - [8. Descripción del proceso](#)
 - [9. Qué debe hacer: los cinco pasos](#)
 - [10. Los tres criterios de aceptación](#)
- [PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ Y CÓMO SE MONTÓ](#)
 - [11. Visión general](#)
 - [12. Cómo se montó en Docker, paso a paso](#)
 - [13. Cómo funciona por dentro \(lo mínimo para entender las pruebas\)](#)
- [PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA CRITERIO](#)
 - [14. Cómo funcionan los laboratorios](#)
 - [15. Laboratorio A — Capacidades: aislado, opcional, adaptador y dos puertas](#)
 - [16. Laboratorio B — Condiciones: aislado, según el método y marca "O"](#)
 - [17. CRITERIO 1 — El registro y la actualización de entradas son operativos](#)
 - [18. CRITERIO 2 — El respaldo y la recuperación se verifican](#)
 - [19. CRITERIO 3 — El camino base opera sin el registro desplegado](#)
 - [20. Pruebas automáticas y recorrido de extremo a extremo](#)
- [PARTE V — PREGUNTAS Y RESPUESTAS](#)
 - [21. Preguntas sobre los conceptos de base](#)
 - [22. Preguntas por lección y por criterio](#)
- [PARTE VI — CÓMO RESPONDER A CADA CRITERIO](#)
 - [23. Redacción modelo de las tres respuestas](#)
 - [24. Lista de comprobación para quien acepta](#)
- [PARTE VII — LÍMITES, RIESGOS Y OPERACIÓN](#)
 - [25. Lo que este informe NO demuestra \(dilo tú primero\)](#)
 - [26. Si algo no sale como en el informe](#)
 - [27. Operación diaria](#)
- [ANEXOS](#)
 - [Anexo A — Los scripts de laboratorio](#)
 - [Anexo B — Autocomprobación del marco conceptual \(con respuestas\)](#)
 - [Anexo C — Ejercicios del método \(con respuesta modelo\)](#)

INFORME FINAL — ERSO 2026-004

Despliegue del registro verificable de datos (VDR) de extensión

Manual de comprensión, construcción y comprobación

Proyecto	Identidad Digital Soberana SSI
ERSo	2026-004 — Despliegue del registro verificable de datos (VDR) de extensión
Desarrollador asignado (según la ERSo)	Geovani Rincón
Pruebas funcionales (según la ERSo)	Luis González
Dominio del laboratorio	civica-desarrollo.avance.org.co
Ubicación del proyecto	/home/geovani/Descargas/generic/bitacora/new_proyect
Salidas de terminal de este informe	Obtenidas ejecutando los comandos reales sobre el proyecto, el 1 de octubre de 2026
Complementos	scripts/lab-004-*.sh (los mismos laboratorios como scripts) · docs/MARCO-CONCEPTUAL.md · informes/ERSo-2026-004.md (versión corta)

Cómo leer este informe

Este documento está escrito para que **cualquier persona** pueda entender qué se pidió, qué se construyó y cómo se comprobó, **sin necesidad de ser experta** en identidad digital, criptografía o contenedores. Cada término técnico se explica la primera vez que aparece, y toda la Parte I es un diccionario ilustrado de esos términos.

Símbolos que verás:

Símbolo	Significa
💡	Idea clave: lo más importante de la sección, en una frase
🔍	Qué vas a ver: lo que debes esperar <i>antes</i> de ejecutar un comando
✅	Qué significa: cómo interpretar lo que el sistema respondió
⚠️	Cuidado: una trampa frecuente o un límite que conviene conocer
🔧	Predice antes de ejecutar: una pregunta para que pienses antes de ver el resultado

Cómo leerlo según tu necesidad:

Si eres...	Lee esto
Quien decide o aprueba	El Resumen ejecutivo y la Parte VII (límites)
Quien necesita entender el problema	Parte I (conceptos) y Parte II (qué pide la ERSO)
Quien debe reproducir y comprobar	Partes III y IV (montaje y laboratorios)
Quien prepara una exposición o defensa	Partes V y VI (preguntas, respuestas y redacción de los criterios)

Cómo están presentados los comandos. Todo lo que verás en bloques `console` es una **sesión de terminal real**: las líneas que empiezan con `$` son lo que se escribe; las demás son lo que el sistema respondió, tal cual. Los valores que cambian en cada ejecución (fechas, identificadores, algunas huellas) aparecerán distintos si repites el ejercicio; lo que debe coincidir es el **patrón** que se explica debajo de cada salida.

Contenido

Parte	Secciones	De qué trata
Resumen ejecutivo	—	El problema, la solución y el resultado en una página
I · Marco conceptual	1 a 4	Todos los términos, explicados en lenguaje sencillo, y el método de trabajo
II · Qué pide la ERSO	5 a 10	Capacidades, condiciones, descripción, pasos y criterios, frase por frase
III · El proyecto	11 a 13	Qué se construyó y cómo se montó en Docker, con la sesión de terminal real
IV · Laboratorios	14 a 20	Las pruebas materiales de cada criterio, con cada comando y su respuesta
V · Preguntas y respuestas	21 y 22	Todas las preguntas de comprensión con su respuesta
VI · Cómo responder	23 y 24	Redacción modelo de cada criterio y lista de comprobación
VII · Límites y operación	25 a 27	Lo que no se demuestra, solución de problemas y operación diaria
Anexos	A a D	Scripts, autocomprobación, ejercicios y referencias

Resumen ejecutivo

El problema. Para que alguien pueda comprobar que una entidad (por ejemplo, un emisor de credenciales) es quien dice ser, necesita encontrar **su clave pública** en un lugar confiable y accesible. Ese lugar se llama *registro verificable de datos* (VDR). Esta ERSO pide construirlo **como una extensión opcional**: que exista, que se pueda encender y apagar, que viva aislado y que, pase lo que pase con él, **no afecte a nada de lo que ya funcionaba**.

La solución. Se construyó un servicio web (programado en Kotlin) con una base de datos PostgreSQL, empaquetado en contenedores Docker detrás de un servidor de entrada (nginx).

Tiene dos "puertas" separadas: una **pública de solo lectura** (cualquiera consulta los documentos) y otra de **escritura protegida** (solo entidades autorizadas pueden publicar). Guarda el **historial completo** e inalterable de cada cambio, permite **respaldar y restaurar** todo su estado y arranca **apagado por defecto**.

El resultado.

#	Criterio de aceptación	Resultado	Prueba material (sección)
1	El registro y la actualización de entradas son operativos (con trazabilidad)	 Cumplido	Laboratorio del criterio 1 (§17)
2	El respaldo y la recuperación se verifican	 Cumplido	Laboratorio del criterio 2 (§18)
3	El camino base opera sin el registro desplegado	 Cumplido	Laboratorio del criterio 3 (§19)

Además se demostraron, con evidencia propia, el **aislamiento** y la **adaptación al método DID**, que el PDF pide pero cuyos criterios no los comprueban de forma directa (§15 y §16).

Lo que conviene saber con honestidad (detalle en la Parte VII). El laboratorio usa una autoridad de certificación **propia** (no una pública), no se desplegó en el dominio real ni se probó desde internet, y el adaptador de método DID solo existe para `did:web`.

PARTE I — MARCO CONCEPTUAL: los términos que necesitas

💡 **Idea clave de esta parte.** Todo el proyecto descansa en pocas ideas. Si las entiendes aquí, el resto del informe es aplicarlas. Cada término está explicado con una frase sencilla, para qué sirve y dónde lo verás en este proyecto.

1. La historia en cinco minutos

Imagina que quieres comprobar que una carta realmente la firmó el rector de una universidad.

- Podrías **llamar al rector** cada vez. Funciona, pero depende de que esté disponible, y si lo hacen millones de personas, es inviable.
- O puedes tener a mano **una muestra confiable de su firma**, y comparar. Para que sirva, esa muestra debe estar en un lugar **público, estable y que nadie pueda alterar a escondidas**.

En el mundo digital pasa lo mismo:

- La "firma" del rector es una **firma digital**, hecha con una **clave privada** que solo él tiene.
- La "muestra confiable de su firma" es su **clave pública**, que cualquiera puede usar para **verificar** (no para falsificar) esa firma.
- El "lugar público, estable y a prueba de alteraciones" es el **registro verificable de datos** (VDR). **Eso es lo que esta ERSO construye.**
- El rector se identifica con un nombre llamado **DID**, y su "muestra de firma" vive en un archivo llamado **DID Document**.

💡 **En una frase:** el VDR es la agenda pública y confiable donde cualquiera busca la clave pública de una entidad.

2. Diccionario de términos

Los términos están agrupados por tema, en el orden en que conviene aprenderlos.

2.1 Criptografía básica

Término	Qué es (en palabras simples)	Para qué sirve	Dónde lo verás aquí
Hash (o huella)	Un "resumen" de longitud fija que se calcula a partir de cualquier contenido. El mismo contenido da siempre el mismo resumen; si cambias una sola letra , el resumen cambia por completo. No se puede deshacer para recuperar el contenido	Comprobar que algo no fue alterado	sha256:cb5799ba... en cada versión de un documento

Término	Qué es (en palabras simples)	Para qué sirve	Dónde lo verás aquí
SHA-256	El método concreto para calcular el hash que usa el proyecto. Produce una huella de 64 caracteres	Lo mismo	El prefijo <code>sha256:</code>
Clave privada	Un número secreto que solo su dueño conoce	Firmar	Nunca aparece en ninguna salida pública
Clave pública	Un número matemáticamente ligado a la privada, que se reparte libremente	Verificar firmas; no sirve para firmar	El <code>publicKeyMultibase</code> del DID Document
Par de claves	La privada y su pública, juntas	Identificarse sin compartir el secreto	Cada entidad tiene su par
Firma digital	El resultado de "firmar" un contenido con la clave privada	Demostrar quién lo firmó y que no cambió	La "prueba de posesión"
Verificar	Comprobar una firma usando la clave pública	Que cualquiera valide sin poder falsificar	El consumidor de DID (otra ERSO)
P-256 / ES256	El tipo de clave (una "curva elíptica") y el algoritmo de firma. Son compactos y seguros	Firmas de 64 bytes	Todas las claves del proyecto
Nonce	Un valor aleatorio de un solo uso	Impedir que una firma capturada se reutilice	El "desafío"
Codificación (Base64url, Base58)	Formas de escribir datos binarios como texto	Meterlos en JSON y URL	Las firmas y las claves
Multikey / multibase	Una convención para escribir una clave pública como texto, indicando su tipo. Las claves P-256 empiezan por <code>zDn</code>	Que cualquiera sepa leer la clave	<code>"publicKeyMultibase": "zDnae..."</code>
JSON canónico	Una forma de escribir un JSON siempre igual (claves ordenadas, sin espacios)	Que el hash y la firma sean reproducibles	El documento publicado

⚠ **Cuatro verbos que se confunden:** *codificar* (cambiar la forma, no es secreto), *hashear* (sacar la huella, no se revierte), *cifrar* (ocultar con una clave) y *firmar* (probar autoría). **Firmar no oculta nada:** el contenido sigue a la vista.

2.2 Redes y canales seguros

Término	Qué es	Para qué sirve	Dónde lo verás
Dominio / DNS	El nombre (<code>civica-desarrollo.avance.org.co</code>) y el sistema que lo traduce a una dirección	Que usemos nombres y no números	El DID es el dominio
HTTP / HTTPS	El protocolo de la web; HTTPS es HTTP protegido con TLS	Pedir y recibir documentos	GET <code>.../did.json</code>

Término	Qué es	Para qué sirve	Dónde lo verás
Puerto	Un número que distingue las "puertas" de una máquina	Separar servicios	8443, 9443, 8444
TLS	La capa que da a HTTPS tres cosas: cifrado (nadie escucha), integridad (nadie altera) y autenticidad del servidor	Comunicación segura	Todo el tráfico
Certificado	Un documento que dice "esta clave pública pertenece a este dominio", firmado por una autoridad	Que el cliente sepa con quién habla	<code>server.crt</code>
CA (autoridad de certificación)	Quien firma certificados; el cliente confía en unas pocas CA	Delegar la confianza	<code>ca.crt</code> (propia de laboratorio)
Cadena de confianza	Certificado → firmado por la CA → en la que el cliente confía	Validar un certificado	<code>openssl verify</code>
mTLS	TLS mutuo : además del servidor, el cliente también presenta certificado	Saber qué entidad está escribiendo	El canal de escritura
Proxy inverso (nginx)	Un servidor que recibe el tráfico y lo reenvía a los servicios internos	Poner TLS, separar canales y esconder el resto	<code>deploy/nginx/nginx.conf</code>
Docker	Una herramienta para ejecutar programas en contenedores aislados	Aislar y repetir el despliegue	Todo el proyecto
Imagen / contenedor	La "receta empaquetada" de un programa / su ejecución	Ejecutar igual en cualquier máquina	<code>vdr-ssi/vdr-service:local</code>
Red de Docker	Una red privada entre contenedores	Que se hablen sin salir al exterior	<code>vdr-ssi_default</code>
Docker Compose	Un archivo que describe todos los contenedores y cómo se conectan	Levantar el proyecto con un comando	<code>deploy/docker-compose.yml</code>
Puerto publicado	Un puerto del contenedor abierto hacia tu máquina	Ser alcanzable desde fuera	Solo nginx publica

2.3 Quién puede escribir (autenticación y permisos)

Término	Qué es	Dónde lo verás
Autenticación	Comprobar quién eres	Certificado y secreto
Autorización	Decidir qué puedes hacer	Cada entidad solo escribe en su espacio
OAuth2 <code>client_credentials</code>	Una máquina cambia su identificador y secreto por un token	<code>POST /admin/v1/oauth/token</code>
Token / JWT	Un permiso temporal (10 minutos) firmado por el servidor	<code>access_token</code>
JWS	El formato <code>cabecera.contenido.firma</code> para firmar datos	Todas las "pruebas"

Término	Qué es	Dónde lo verás
Desafío (<i>challenge</i>)	El servidor entrega un nonce y pide que se firme	POST /challenges
Prueba de posesión	Demostrar que posees la clave privada (no solo que sabes algo) firmando el desafío	Una escritura válida
Replay (repetición)	Reenviar algo que se capturó antes	El ataque que frena el nonce de un solo uso
Idempotencia	Repetir una petición no duplica el efecto	Cabecera Idempotency-Key
Concurrencia optimista	"Escribo solo si la versión actual es la N"	Cabecera If-Match
Namespace	El espacio del DID reservado a una entidad (entidades/avance)	Permisos por entidad

Códigos HTTP que aparecen (la respuesta numérica del servidor):

Código	Significa	Ejemplo aquí
200 / 201	Bien / creado	Escritura confirmada
400	Solicitud mal formada	nginx exige certificado cliente y no lo recibe
401	No sé quién eres	Falta el token
403	Sé quién eres, pero no tienes permiso	Firma inválida
404	No existe	DID inexistente, o ruta que no existe
409	Conflicto con el estado actual	Crear un DID que ya existe, o restaurar sobre datos vivos
410	Existió, pero ya no (a propósito)	DID desactivado
412	Falla una precondition	If-Match con la versión equivocada
422	Entendible pero inválido	Respaldo alterado, documento que no cumple el perfil
502	El servidor de entrada no encuentra al de atrás	nginx cuando el VDR está detenido

2.4 Identidad descentralizada

Término	Qué es	Dónde lo verás
Identificador	Un nombre único para algo	—
DID	Un identificador con la forma did:<método>:<resto> que su dueño controla sin una autoridad central	did:web:civica-desarrollo.avance.org.co:entidades:avance
Método DID	La "receta" que dice dónde y cómo viven los DID de ese tipo (en una web, en una cadena de bloques...)	La palabra web

Término	Qué es	Dónde lo verás
<code>did:web</code>	El método cuyo registro es un servidor web : el DID se convierte en una dirección	Todo el proyecto
DID Document	Un archivo JSON público con las claves públicas del DID y para qué sirve cada una	<code>.../did.json</code>
<code>verificationMethod</code>	La lista de claves públicas dentro del documento	<code>"type": "Multikey"</code>
<code>authentication</code>	Qué claves sirven para identificarse	Dentro del documento
<code>assertionMethod</code>	Qué claves sirven para firmar afirmaciones (por ejemplo, credenciales)	Dentro del documento
DID URL	Un DID más <code>#algo</code> , que apunta a una parte	<code>...#key-1</code>
Resolver	Dado un DID, obtener su documento	Otra ERSO (consumidor)
VDR	<i>Verifiable Data Registry</i> : el lugar donde se publican y consultan los DID Documents. Es un rol, no una tecnología	<code>vdr-service</code>

💡 **Tres errores comunes:** un DID **no es** necesariamente una cadena de bloques; un DID Document **no es** una credencial (dice qué claves tiene un identificador, no datos de una persona); y poseer una clave **no equivale** a ser una persona.

2.5 Datos, historial y recuperación

Término	Qué es	Dónde lo verás
PostgreSQL	La base de datos que guarda el estado	Contenedor <code>postgres</code>
Tabla / fila / clave primaria	La estructura de los datos / un registro / el campo que lo identifica	<code>did_documents</code> (clave: el DID)
Transacción	Un conjunto de cambios que se aplican todos o ninguno	Nunca queda el registro a medias
Estado actual e historial	La "foto de hoy" y el "álbum" de todas las versiones	<code>did_documents</code> y <code>did_document_versions</code>
Versión	Un número que crece con cada cambio	<code>v1</code> , <code>v2</code> ...
Append-only (solo se agrega)	Una tabla donde nada se edita ni se borra	El historial
Trigger	Una regla dentro de la base que se dispara sola	Lo que bloquea <code>DELETE</code> y <code>UPDATE</code>
Auditoría	El diario de quién hizo qué y cuándo, incluidos los intentos rechazados	<code>audit_log</code>

Término	Qué es	Dónde lo verás
Respaldo (backup)	Una copia completa del estado, guardada fuera	POST /admin/v1/backup
Checksum	El hash de un archivo completo	Detecta que un respaldo fue alterado
Restauración	Volver a cargar un respaldo	POST /admin/v1/restore
TRUNCATE	Vaciar una tabla por completo	Para simular la pérdida total

2.6 Palabras propias de las ERSO

Término	Qué es
ERSo	<i>Especificación de Requisitos de Software</i> : el documento que dice qué debe cumplir un desarrollo (capacidades, condiciones, pasos, criterios)
Capacidad	Algo que el sistema debe poder hacer
Condición	Una restricción que el diseño debe respetar siempre
Criterio de aceptación	Una condición medible que se debe cumplir, con la evidencia que la demuestra
Camino base	Lo que el piloto hace sin la extensión; debe seguir funcionando siempre
Marca "O"	<i>Opcional</i> : fuera del alcance base y desactivada por defecto
Regresión	Comprobar que lo que ya funcionaba sigue funcionando tras un cambio
Falla cerrada (<i>fail-closed</i>)	Ante la duda, lo riesgoso queda apagado o se rechaza
Falla temprana (<i>fail-fast</i>)	Es mejor no arrancar con una configuración peligrosa que arrancar y avisar
Prueba automática	Un programa que comprueba otro programa; repetible y rápido
Prueba de extremo a extremo (E2E)	Una prueba que recorre el sistema real (con red, TLS y certificados)
Evidencia	Lo que demuestra un criterio: una prueba, un registro, una salida de terminal

3. Las siete ideas madre

#	Idea	En una frase
1	Confianza por verificación, no por autoridad	En lugar de "confío porque lo dice X", "confío porque puedo comprobarlo"
2	La pública verifica; la privada firma	La privada jamás sale de su dueño
3	Un hash es una huella	Cambia un byte y la huella cambia por completo
4	Leer es libre; cambiar exige demostrar	Cualquiera lee un documento; escribirlo exige canal seguro, permiso y prueba de la clave
5	Defensa en capas	Si una capa falla, quedan las otras

#	Idea	En una frase
6	Cada rol tiene su clave y su canal	Nadie comparte puertas
7	Lo opcional está apagado y aislado	La extensión no puede romper lo que ya funcionaba

4. El método de trabajo: cómo se piensa un criterio

Para pasar de una frase del PDF a una prueba que otra persona pueda repetir, se siguió siempre el mismo ciclo, en dos niveles.

Para aprender cada tema (cinco pasos): *concepto* → *observación* → *predicción* → *correlación* → *evidencia*. Primero entiendes la idea, luego la ves funcionar, **predices** qué pasará antes de mirar (aquí es donde se aprende de verdad), conectas lo visto con el diseño y, al final, sabes demostrarlo.

Para resolver un criterio (siete pasos):

#	Paso	La pregunta
1	Descomponer	¿Cuál es el verbo, el objeto y la evidencia que pide?
2	Hacerlo comprobable	¿Qué debería ser cierto y cómo lo vería un tercero?
3	Buscar el caso negativo	¿Cómo intentaría romperlo un atacante o un error?
4	Elegir el mecanismo	¿Qué pieza lo garantiza? ¿Puede ser una capa baja (base de datos, proxy)?
5	Implementar y probar a la vez	¿Cómo falla esta prueba si el mecanismo no existe?
6	Doble evidencia	Prueba automática y prueba por la red real
7	Declarar los límites	¿Qué no se probó o está simulado?

💡 **El verbo manda.** "Se rechaza" pide una prueba negativa. "Queda registrada" pide mostrar el dato guardado. "Opera sin..." pide una regresión. Y **sin caso negativo, lo que tienes es una demostración, no una prueba.**

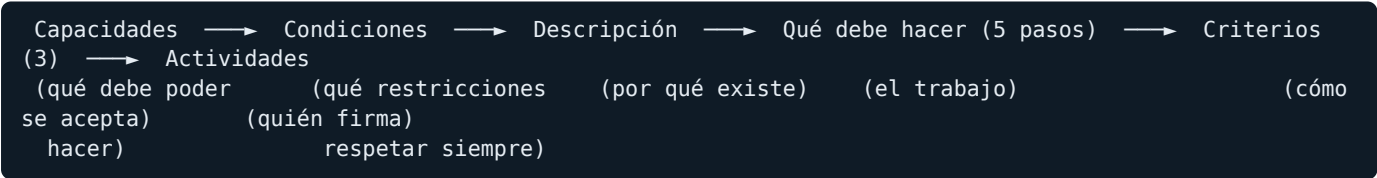
PARTE II — QUÉ PIDE LA ERSo 004, EXPLICADO FRASE POR FRASE

5. La ficha del documento

Campo del PDF	Valor
Proyecto	Identidad Digital Soberana SSI
Título	Despliegue del registro verificable de datos (VDR) de extensión
Desarrollador	Geovani Rincón
Fecha de la ERSo	Viernes, 18 de septiembre de 2026
Fecha de desarrollo	Lunes, 21 de septiembre de 2026, 7:30 a. m.
Fecha de prueba	Miércoles, 23 de septiembre de 2026, 7:30 a. m.
Responsables de las actividades	Análisis y diseño: Karen Flórez Madiedo · Asignar desarrollador y archivo: Rocío Villamizar · Implementar: Geovani Rincón · Plantilla y pruebas funcionales: Luis González

⚠ Las firmas y fechas de la tabla de actividades del PDF **corresponden a las personas responsables**; este informe aporta las **evidencias técnicas** que ellas necesitan, no las firmas.

El PDF se organiza en seis bloques, y este informe los recorre en el mismo orden:



6. Capacidades del proceso

6.1 La frase principal, desarmada

"Se requiere habilitar **de forma opcional** los **endpoints** y el **adaptador** de registro verificable de datos que soportan la **publicación del DID Document**, en un **entorno aislado** y **sin efecto sobre el camino base**."

#	Fragmento	Qué significa en lenguaje sencillo	Cómo se materializó
1	"de forma opcional"	La extensión no es obligatoria : se enciende o se apaga, y por defecto está apagada	Una variable de configuración, <code>VDR_ENABLED</code> , que vale <code>false</code> si no se define

#	Fragmento	Qué significa en lenguaje sencillo	Cómo se materializó
2	"los endpoints"	Las "puertas" web por donde se lee y se escribe el registro	Un canal público de lectura y un canal de escritura protegido
3	"el adaptador"	Una pieza que traduce entre un DID y la forma concreta en que se guarda su documento. Existe porque el registro no es igual para todos los métodos DID	La clase <code>DidWebAdapter</code>
4	"registro verificable de datos... publicación del DID Document"	El propósito: ser el lugar público donde se publican y se consultan los documentos	El servicio <code>vdr-service</code> y la base PostgreSQL
5	"entorno aislado"	La extensión vive en su propio espacio, con una sola puerta de entrada	Contenedores Docker; solo nginx publica puertos
6	"sin efecto sobre el camino base"	El resto del sistema sigue igual con la extensión encendida, apagada o rota	Con la opción apagada ni siquiera se conecta a la base de datos

💡 **Qué se busca, en una frase:** que exista un lugar donde publicar DID Documents, que se pueda **encender y apagar**, que viva **aislado** y que, esté como esté, **no afecte a nada de lo que ya funcionaba**.

Lo que la frase no dice, y conviene notar: no menciona cadenas de bloques, ni qué método DID usar, ni qué tecnología. Pide *comportamientos* (opcional, aislado, inocuo), no una tecnología concreta.

6.2 Las cuatro capacidades

"Entre las capacidades del proceso se encuentran: exponer los endpoints de registro de identificadores; adaptar la forma del registro al método DID empleado; registrar y actualizar entradas en el registro; respaldar y recuperar el estado del registro."

#	Capacidad	Qué se busca	Cómo se resolvió	Criterio que la comprueba
1	Exponer los endpoints de registro	Que haya puertas HTTP para publicar y consultar	<code>GET */did.json</code> (público) y <code>/admin/v1/...</code> (escritura protegida)	Se usan en los criterios 1 y 2
2	Adaptar la forma del registro al método DID	Que el servicio sepa cómo es el registro de <code>did:web</code> sin mezclar eso con el resto	<code>DidMethodAdapter</code> / <code>DidWebAdapter</code> : DID $\neq$ dirección web	Indirecto (prueba paso 2)
3	Registrar y actualizar entradas	Crear documentos y cambiarlos, con control	<code>PUT /admin/v1/documents/{did}</code> con versión esperada	Criterio 1
4	Respaldar y recuperar el estado	Poder reconstruir el registro tras perderlo	<code>POST /admin/v1/backup</code> y <code>/restore</code>	Criterio 2

7. Condiciones del proceso

Una **condición** no es algo que el sistema *haga*: es una **restricción** que el diseño debe respetar *siempre*. Para cada una conviene preguntarse **qué problema evita**.

Condición 1 — "El despliegue se realiza en entorno aislado, sin afectar el camino base."

- **Qué significa.** Lo nuevo vive en su propio espacio. Si falla, se apaga o se actualiza, **nada de lo que ya funcionaba se entera**.
- **Qué problema evita.** Un componente opcional que, al romperse, tumba todo lo demás.
- **Dos clases de aislamiento:**

Aislado en...	Qué es	Cómo se logra
Red	Nadie de fuera llega directo a la aplicación ni a la base de datos	Solo nginx publica puertos; el resto vive en una red privada de Docker
Fallo	Si el VDR cae, los demás servicios siguen	Cada servicio es un contenedor independiente

💡 *Aislado no es "inexistente"*: desde dentro de la red privada se llega a todo; lo que no se puede es llegar desde fuera saltándose nginx.

Condición 2 — "La forma del registro depende del método DID y no implica siempre una cadena de bloques."

- **Qué significa.** Un **método DID** define *dónde* y *cómo* viven los documentos. Cada método tiene su propia "forma" de registro:

Método	La "forma" del registro
did:web	Una dirección web: el DID es una URL
did:ebssi	Una infraestructura europea basada en cadena de bloques
did:key	Ninguna: la clave va dentro del propio identificador

- **Qué problema evita.** Creer que "registro verificable" = cadena de bloques, y construir algo enorme e innecesario. Aquí el registro es **un servicio web indexado por identificador**.
- ⚠️ **Una precisión honesta.** El adaptador es el sitio correcto para aislar el método, pero hoy la interfaz devuelve un tipo llamado `DidWebId`, es decir, **todavía tiene forma de did:web**. Para agregar otro método **no basta con escribir otro adaptador**: habría que generalizar también ese tipo. Solo existe `did:web`.

Condición 3 — "Componente con marca O: opción fuera del alcance base, desactivada por defecto."

- **Qué significa.** "O" = **opcional**. No forma parte del piloto base y **arranca apagado** salvo que alguien lo encienda a propósito.

- **Qué problema evita.** (1) Desplegar el VDR sin querer y abrir superficie de ataque; (2) que el piloto base dependa de algo no aprobado; (3) que un error al escribir la configuración (`si` , `True`) **encienda** algo por accidente.
- **Dos reglas que lo materializan:**

Regla	Qué hace	Nombre del principio
Solo <code>true</code> exacto enciende	<code>si</code> , <code>True</code> , <code>1</code> o un espacio de más ⇒ apagado	<i>Falla cerrada</i>
Encendido sin secreto ni clientes ⇒ no arranca	Es mejor un error en el despliegue que un servicio inseguro funcionando	<i>Falla temprana</i>

8. Descripción del proceso

Es el **porqué** de la ERSO. Tiene tres frases importantes.

"El piloto puede operar sin un registro público obligatorio; este registro es una extensión opcional que permite publicar y gestionar documentos de identificador más allá del camino base."

- El piloto **ya funciona sin esto**: la ERSO **agrega**, no repara.
- "Publicar **y gestionar**": no es solo subir un archivo; es crear, actualizar, trazar, respaldar.

"El registro no es necesariamente una cadena de bloques: puede ser un servicio web indexado por identificador, y su forma depende del método DID elegido."

- "Indexado por identificador": en la tabla `did_documents` el DID es la **clave primaria**; dado un DID, el servicio encuentra su documento directamente, como en un diccionario.

"Literatura y temas a consultar" — la lista de estudio que asigna la ERSO:

Lectura	Qué te aporta	Para qué la usarías
DID Resolution v1 (W3C)	Define <i>resolver</i> (DID → documento) y los errores normativos (<code>notFound</code> , <code>invalidDid</code> ...)	La lectura pública y el consumidor
DIDs v1.1 (W3C), métodos y registros	Qué es un método y que cada uno define su registro	Justifica el adaptador
Verifiable Data Registry (modelo de credenciales)	El rol abstracto, no una tecnología	Responder "¿qué es un VDR?"
REST de "managed documents"	El patrón: documentos con versión, If-Match , historial	Diseño de los endpoints
Respaldo y restauración de estado	Cómo exportar y recargar estado sin corromperlo	Criterio 2

Los enlaces están en `docs/VERSIONES-NORMATIVAS.md` .

9. Qué debe hacer: los cinco pasos

Paso	Lo que pide el PDF	Capacidad / condición asociada	Criterio que lo comprueba
1	Desplegar el adaptador y los endpoints en un entorno aislado, sin tocar el camino base	Capacidad 1 · Condición 1	Ninguno directo
2	Adaptar la forma del registro al método DID empleado	Capacidad 2 · Condición 2	Ninguno directo
3	Implementar el registro y la actualización de entradas	Capacidad 3	Criterio 1
4	Implementar el respaldo y la recuperación del estado	Capacidad 4	Criterio 2
5	Asegurar que el sistema base sigue operando con el registro apagado	Condiciones 1 y 3	Criterio 3

 **Observación importante.** Los pasos 1 y 2 **no tienen un criterio propio**. Por eso, además de los tres criterios, este informe aporta **evidencia adicional** que los demuestra (laboratorios de las secciones 15 y 16). Es lo que se llama declarar con transparencia qué cubre el criterio y qué cubre la evidencia propia.

10. Los tres criterios de aceptación

Criterio 1. "El registro y la actualización de entradas son operativos; evidencia: pruebas de registro, actualización y trazabilidad."

Criterio 2. "El respaldo y la recuperación del registro se verifican; evidencia: prueba de respaldo y restauración."

Criterio 3. "El camino base opera sin el registro desplegado; evidencia: pruebas de regresión con la opción apagada."

Aplicando los primeros pasos de la receta (descomponer, hacerlo comprobable, buscar el caso negativo):

Criterio	El verbo	Lo que debo poder mostrar	El caso negativo (intento de romperlo)	Mecanismo que lo garantiza
1	<i>Son operativos</i>	Crear, actualizar y ver el historial	Actualizar con la versión equivocada; usar una clave ajena; borrar el historial	If-Match, prueba de posesión, tabla de solo-agregar
2	<i>Se verifican</i>	Respaldar, perder todo, restaurar y comparar	Restaurar sobre datos vivos; restaurar un respaldo alterado	Checksum; restaurar solo sobre un registro vacío; verificación posterior
3	<i>Opera sin</i>	Que el camino base responda con la opción apagada	Que la aplicación se conecte a la base o abra el canal de escritura estando apagada	Las rutas del registro no se registran si está apagado

Fíjate en una sutileza del criterio 3: dice "sin el registro **desplegado**", no "apagado". Se cubren **las dos cosas**: con el servicio levantado y la opción apagada, y sin base de datos en absoluto (las pruebas de ese criterio no necesitan PostgreSQL).

PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ Y CÓMO SE MONTÓ

11. Visión general

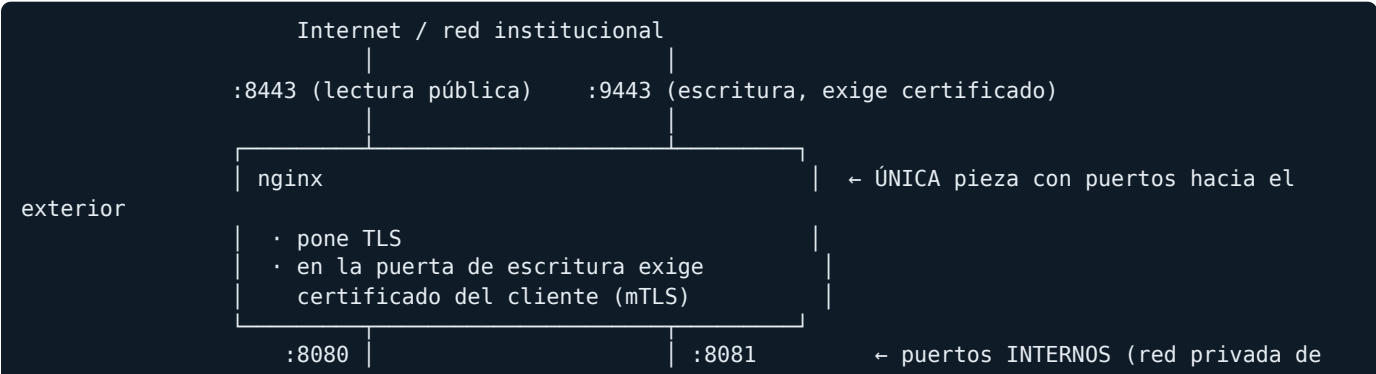
Para cumplir la ERSo 004 se desarrolló un proyecto completo y reproducible. En una frase: **un servicio web con base de datos, empaquetado en contenedores, con dos puertas separadas y arrancado apagado por defecto.**

11.1 Las piezas y lo que hace cada una

Pieza	Qué es	Qué hace en la ERSo 004
vdr-service	El programa principal (Kotlin / Ktor)	Es el registro: publica los DID Documents, recibe las escrituras autorizadas, guarda el historial, respalda y restaura
PostgreSQL	La base de datos	Guarda el estado, el historial y la auditoría. Nadie de fuera la ve
nginx	El servidor de entrada (proxy inverso)	Pone TLS, separa las dos puertas y exige certificado cliente en la de escritura. Es la única pieza con puertos hacia el exterior
did-core	Una librería	Las reglas del DID: convertir un did:web en dirección, validar documentos, firmar y verificar
did-resolver	Otra librería	El consumidor de solo lectura (otra ERSo); aquí sirve para comprobar
did-tools	Un programa de línea de comandos	Actúa como "la entidad": genera claves, arma documentos, escribe en el registro. Es lo que usamos en los laboratorios
Docker Compose	El orquestador	Levanta todo con un comando y define qué contenedores existen y cómo se conectan

El proyecto contiene además componentes de otras ERSo (cartera, credenciales). **No se tratan en este informe**; solo importa que comparten el mismo `docker-compose.yml`.

11.2 El diagrama





💡 **Lo esencial:** hay **dos puertas** (lectura pública y escritura protegida) y **una sola entrada al exterior** (nginx). La aplicación y la base de datos **no se ven desde fuera**.

11.3 Las carpetas del proyecto

Carpeta	Contenido
vdr-service/	El registro: rutas HTTP, lógica, esquema SQL y pruebas automáticas
did-core/ , did-resolver/ , did-tools/	Las librerías y la herramienta de línea de comandos
deploy/	Todo lo necesario para desplegar: <code>docker-compose.yml</code> , <code>Dockerfile</code> , <code>nginx/nginx.conf</code> , <code>certs/</code> (certificados) y <code>.env</code> (secretos)
scripts/	Generadores de certificados y secretos, el recorrido de extremo a extremo (<code>e2e.sh</code>) y los laboratorios (<code>lab-*.sh</code>)
docs/	Marco conceptual, guía de exposición, versiones normativas
informes/	Un informe por ERSO (este documento amplía el de la 004)
evidencias/	Los registros de las corridas de prueba

12. Cómo se montó en Docker, paso a paso

Esta sección es una **sesión de terminal real**: se muestra cada comando y lo que el sistema respondió.

🔍 **Qué vas a hacer.** Partir de la carpeta del proyecto y llegar a tener el registro funcionando con cuatro piezas: PostgreSQL, el servicio, nginx y las herramientas.

12.1 Ubicarse en el proyecto

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect && ls
build
build.gradle.kts
credentials-core
credential-service
deploy
did-core
did-resolver
did-tools
docs
ersos
evidencias
gradle
```

```
gradle.properties
gradlew
gradlew.bat
informes
README2.md
README.md
scripts
settings.gradle.kts
vdr-service
wallet-core
wallet-service
wallet-sim
$ ls deploy
certs
docker-compose.yml
Dockerfile
nginx
```

✓ **Qué significa.** Ves las carpetas de la tabla 11.3. En `deploy/` están el `docker-compose.yml`, el `Dockerfile` (la receta para empaquetar un programa en una imagen) y la configuración de nginx.

12.2 Crear los certificados de laboratorio

💡 **Para qué.** TLS necesita certificados. En un entorno real los emitiría una autoridad pública; en el laboratorio creamos **una autoridad propia** y con ella firmamos todo.

```
$ bash scripts/gen-dev-certs.sh
Ya existen certificados en deploy/certs (bórrelos para regenerar).
$ bash scripts/gen-env.sh
deploy/.env ya está actualizado.
$ ls deploy/certs
avance-issuer.crt
avance-issuer.key
avance-issuer.p12
ca.crt
ca-falsa.crt
ca-falsa.key
ca.key
intruso.crt
intruso.key
intruso.p12
jwt.sig
lab-operator.crt
lab-operator.key
lab-operator.p12
server.crt
server.key
vdr-admin.crt
vdr-admin.key
vdr-admin.p12
wallet-backend.crt
wallet-backend.key
wallet-backend.p12
```

Comando	Qué hace	Qué se busca
<code>gen-dev-certs.sh</code>	Crea la CA de laboratorio , el certificado del servidor y un certificado cliente por entidad. Es idempotente : si ya existen, no los toca	Tener con qué hacer TLS y mTLS

Comando	Qué hace	Qué se busca
<code>gen-env.sh</code>	Crea <code>deploy/.env</code> con secretos aleatorios (contraseña de la base, secreto del token, secretos de cada entidad). Si ya existe, no cambia los secretos	No escribir contraseñas a mano
<code>ls</code> <code>deploy/certs</code>	Lista lo creado	Comprobarlo

✅ **Qué significa.** El mensaje "Ya existen..." indica que esto ya se hizo antes y el script **respetó** lo existente. En `certs/` verás:

Archivo	Qué es	¿Secreto?
<code>ca.crt</code> / <code>ca.key</code>	La CA de laboratorio y su clave	<code>ca.key</code> sí
<code>server.crt</code> / <code>server.key</code>	Certificado TLS del servidor	<code>server.key</code> sí
<code>vdr-admin.*</code> , <code>avance-issuer.*</code> , <code>lab-operator.*</code>	Certificado cliente de cada entidad (el <code>.p12</code> es el mismo par empaquetado)	<code>.key</code> y <code>.p12</code> sí
<code>ca-falsa.*</code> e <code>intruso.*</code>	Una CA que no es la nuestra y un certificado firmado por ella: sirven para las pruebas negativas	—

⚠️ Estos archivos **no se suben a ningún repositorio** y no son para producción: allí se usarían una CA reconocida y un gestor de secretos.

12.3 Compilar el programa

```
$ ./gradlew installDist 2>&1 | tail -3
BUILD SUCCESSFUL in 993ms
30 actionable tasks: 30 up-to-date
Consider enabling configuration cache to speed up this build:
https://docs.gradle.org/9.4.1/userguide/configuration_cache_enabling.html
```

Parte	Qué hace
<code>./gradlew</code>	El "constructor" del proyecto (Gradle), que descarga lo necesario
<code>installDist</code>	Compila el código y deja los programas listos en <code>*/build/install/</code>
<code>2>&1 tail -3</code>	Muestra solo las últimas tres líneas del resultado

✅ **Qué significa.** `BUILD SUCCESSFUL` = el código compila. Si hubiera un error de programación, aquí aparecería.

12.4 Construir las imágenes de Docker

```
$ cd deploy
$ set -a; . ./env; set +a; export LOCAL_UID=$(id -u) LOCAL_GID=$(id -g)
$ docker compose --profile tools build 2>&1 | tail -4
Image vdr-ssi/vdr-service:local Built
Image vdr-ssi/wallet-service:local Built
Image vdr-ssi/credential-service:local Built
Image vdr-ssi/holder-sim:local Built
```

Comando	Qué hace
<code>set -a; . ./env; set +a</code>	Carga en el terminal las variables de <code>deploy/.env</code> (dominio, contraseñas...). <code>set -a</code> hace que queden disponibles para los programas que lances
<code>export LOCAL_UID=... LOCAL_GID=...</code>	Para que los archivos que cree la herramienta dentro del contenedor te pertenezcan
<code>docker compose -- profile tools build</code>	Construye las imágenes (los paquetes ejecutables) de todos los servicios, incluida la herramienta de línea de comandos (<code>--profile tools</code>)

✓ **Qué significa.** Líneas como `Built / done` indican que cada imagen quedó lista. Una **imagen** es la receta empaquetada; un **contenedor** es esa receta en ejecución.

12.5 Ver qué servicios define el proyecto y arrancarlos

```
$ docker compose config --services
credential
postgres
vdr
nginx
wallet
$ docker compose down
Container vdr-ssi-credential-1 Stopping
Container vdr-ssi-nginx-1 Stopping
Container vdr-ssi-wallet-1 Stopping
Container vdr-ssi-wallet-1 Stopped
Container vdr-ssi-wallet-1 Removing
Container vdr-ssi-credential-1 Stopped
Container vdr-ssi-credential-1 Removing
Container vdr-ssi-wallet-1 Removed
Container vdr-ssi-credential-1 Removed
Container vdr-ssi-nginx-1 Stopped
Container vdr-ssi-nginx-1 Removing
Container vdr-ssi-nginx-1 Removed
Container vdr-ssi-vdr-1 Stopping
Container vdr-ssi-vdr-1 Stopped
Container vdr-ssi-vdr-1 Removing
Container vdr-ssi-vdr-1 Removed
Container vdr-ssi-postgres-1 Stopping
Container vdr-ssi-postgres-1 Stopped
Container vdr-ssi-postgres-1 Removing
Container vdr-ssi-postgres-1 Removed
Network vdr-ssi_default Removing
Network vdr-ssi_default Removed
$ docker compose up -d
Network vdr-ssi_default Creating
Network vdr-ssi_default Creating
Network vdr-ssi_default Created
Network vdr-ssi_default Created
Container vdr-ssi-credential-1 Creating
Container vdr-ssi-postgres-1 Creating
Container vdr-ssi-credential-1 Created
Container vdr-ssi-postgres-1 Created
Container vdr-ssi-vdr-1 Creating
Container vdr-ssi-wallet-1 Creating
Container vdr-ssi-wallet-1 Created
Container vdr-ssi-vdr-1 Created
Container vdr-ssi-nginx-1 Creating
Container vdr-ssi-nginx-1 Created
Container vdr-ssi-postgres-1 Starting
Container vdr-ssi-credential-1 Starting
Container vdr-ssi-postgres-1 Started
Container vdr-ssi-postgres-1 Waiting
```

```

Container vdr-ssi-postgres-1 Waiting
Container vdr-ssi-credential-1 Started
Container vdr-ssi-postgres-1 Healthy
Container vdr-ssi-vdr-1 Starting
Container vdr-ssi-postgres-1 Healthy
Container vdr-ssi-wallet-1 Starting
Container vdr-ssi-vdr-1 Started
Container vdr-ssi-nginx-1 Starting
Container vdr-ssi-wallet-1 Started
Container vdr-ssi-nginx-1 Started

```

Comando	Qué hace
<code>config --services</code>	Lista los servicios que declara <code>docker-compose.yml</code>
<code>down</code>	Detiene y elimina los contenedores y la red. Los datos de PostgreSQL se conservan porque viven en un <i>volumen</i> aparte
<code>up -d</code>	Crea y arranca todos los servicios, en segundo plano (<code>-d</code>)

✓ **Qué significa.** Cada línea `Container ... Started` es un servicio arrancando. `Healthy` en PostgreSQL significa que la base ya acepta conexiones: el resto **espera** a que esté lista.

12.6 Comprobar que está vivo

Definimos unos **atajos** para no repetir opciones largas, y esperamos a que responda:

```

$ D=$VDR_DOMAIN; PUB="--cacert certs/ca.crt --resolve $D:8443:127.0.0.1"; ADM="--cacert certs/ca.crt
--resolve $D:9443:127.0.0.1"; CRED="--cacert certs/ca.crt --resolve $D:8444:127.0.0.1";
W=./evidencias/work
$ PSQL="docker compose exec -T postgres psql -U $POSTGRES_USER -d $POSTGRES_DB"
$ for i in $(seq 1 40); do curl -sf $PUB https://$D:8443/health >/dev/null && break; sleep 1; done
$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE
CREATED            STATUS              PORTS
vdr-ssi-credential-1 vdr-ssi/credential-service:local  "/bin/sh -c 'exec $A..."  credential  5
seconds ago      Up 5 seconds
vdr-ssi-nginx-1      nginx:1.27-alpine      "/docker-entrypoint..."  nginx      5
seconds ago      Up 1 second          80/tcp, 0.0.0.0:8444->8444/tcp, [::]:8444->8444/tcp,
0.0.0.0:8443->443/tcp, [::]:8443->443/tcp, 0.0.0.0:9443->8443/tcp, [::]:9443->8443/tcp
vdr-ssi-postgres-1   postgres:16-alpine      "docker-entrypoint.s..."  postgres   5
seconds ago      Up 5 seconds (healthy)  5432/tcp
vdr-ssi-vdr-1        vdr-ssi/vdr-service:local  "/bin/sh -c 'exec $A..."  vdr        5
seconds ago      Up 1 second
vdr-ssi-wallet-1     vdr-ssi/wallet-service:local  "/bin/sh -c 'exec $A..."  wallet     5
seconds ago      Up 1 second
$ curl -s $PUB https://$D:8443/health
{"status":"UP","vdr":"enabled"}

```

Atajo	Qué guarda	Para qué
<code>D</code>	El dominio del laboratorio	No repetirlo
<code>PUB</code>	Las opciones de <code>curl</code> para la puerta pública	<code>--cacert</code> : "confía en nuestra CA"; <code>--resolve</code> : "ese dominio, en ese puerto, está en mi máquina (127.0.0.1)"
<code>ADM</code>	Lo mismo para la puerta de escritura (puerto 9443)	

Atajo	Qué guarda	Para qué
CRED	Lo mismo para el canal de cartera (puerto 8444)	Solo se usa en el laboratorio de condiciones
PSQL	Un atajo para ejecutar <code>psql</code> (el cliente de PostgreSQL) dentro del contenedor de la base	Consultar la base

💡 **Por qué `--resolve`.** El nombre `civica-desarrollo.avance.org.co` aún no apunta a tu máquina en el DNS real, pero el certificado solo es válido para ese **nombre**. Con `--resolve` usamos el nombre correcto y llegamos a tu máquina.

✅ Qué significa.

- `docker compose ps` lista los servicios y sus puertos. Fíjate: **solo** `nginx` muestra flechas `0.0.0.0:...->...` (puertos publicados). `postgres` muestra `5432/tcp` **sin flecha**: el contenedor lo tiene abierto, pero **no está publicado** hacia tu máquina.
- La respuesta de `/health` es `{"status":"UP","vdr":"enabled"}`: el servicio está vivo y la extensión **encendida**.

12.7 Comprobar la configuración de nginx y sus puertas

```
$ docker compose exec -T nginx nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
$ grep -nE 'listen|server_name' nginx/nginx.conf
19:     listen 443 ssl;
20:     server_name civica-desarrollo.avance.org.co localhost;
39:     listen 8443 ssl;
40:     server_name civica-desarrollo.avance.org.co localhost;
65:     listen 8444 ssl;
66:     server_name civica-desarrollo.avance.org.co localhost;
```

✅ **Qué significa.** `syntax is ok / test is successful`: la configuración es válida. Las líneas `listen` muestran las **puertas** de nginx dentro del contenedor: `443` (lectura pública), `8443` (escritura con certificado cliente) y `8444` (otro canal, de otra ERSO). En tu máquina, Docker los publica como `8443`, `9443` y `8444`.

12.8 La base de datos: tablas y la regla que protege el historial

```
$ echo "\dt" | $PSQL
      List of relations
Schema |          Name          | Type  | Owner
-----+-----+-----+-----
public | audit_log              | table | vdr
public | challenges              | table | vdr
public | did_document_versions  | table | vdr
public | did_documents          | table | vdr
public | entity_accounts        | table | vdr
public | namespaces              | table | vdr
public | operations              | table | vdr
public | schema_migrations      | table | vdr
public | wallet_audit            | table | vdr
public | wallet_challenges      | table | vdr
public | wallet_citizens         | table | vdr
public | wallet_did_backups      | table | vdr
public | wallet_did_publications | table | vdr
```

```

public | wallet_instances      | table | vdr
public | wallet_recovery_tokens    | table | vdr
(15 rows)
$ echo "\d did_document_versions" | $PSQL
               Table "public.did_document_versions"
   Column   |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
 did        | text           |           | not null |
 version    | integer        |           | not null |
 operation   | text           |           | not null |
 document    | text           |           |          |
 hash       | text           |           | not null |
 actor      | text           |           | not null |
 operation_id | uuid           |           | not null |
 created_at  | timestamp with time zone |           | not null | now()
Indexes:
    "did_document_versions_pkey" PRIMARY KEY, btree (did, version)
    "versions_at_idx" btree (did, created_at)
Check constraints:
    "did_document_versions_operation_check" CHECK (operation = ANY (ARRAY['CREATE'::text, 'UPDATE'::text, 'DEACTIVATE'::text]))
Foreign-key constraints:
    "did_document_versions_did_fkey" FOREIGN KEY (did) REFERENCES did_documents(did)
Triggers:
    versions_append_only BEFORE DELETE OR UPDATE ON did_document_versions FOR EACH ROW EXECUTE FUNCTION forbid_mutation()
$ echo "select tgname as trigger, relname as tabla from pg_trigger t join pg_class c on c.oid=t.tgrelid where not tgisinternal" | $PSQL
   trigger   |      tabla
-----+-----
 versions_append_only | did_document_versions
 audit_append_only   | audit_log
 wallet_audit_append_only | wallet_audit
(3 rows)

```

Comando	Qué hace	Qué se busca
\dt	Lista las tablas	Ver dónde vive cada cosa
\d did_document_versions	Describe la tabla del historial	Ver sus columnas y los <i>triggers</i>
la consulta a pg_trigger	Lista las reglas automáticas de la base	Confirmar que existen las que protegen el historial y la auditoría

✓ Qué significa.

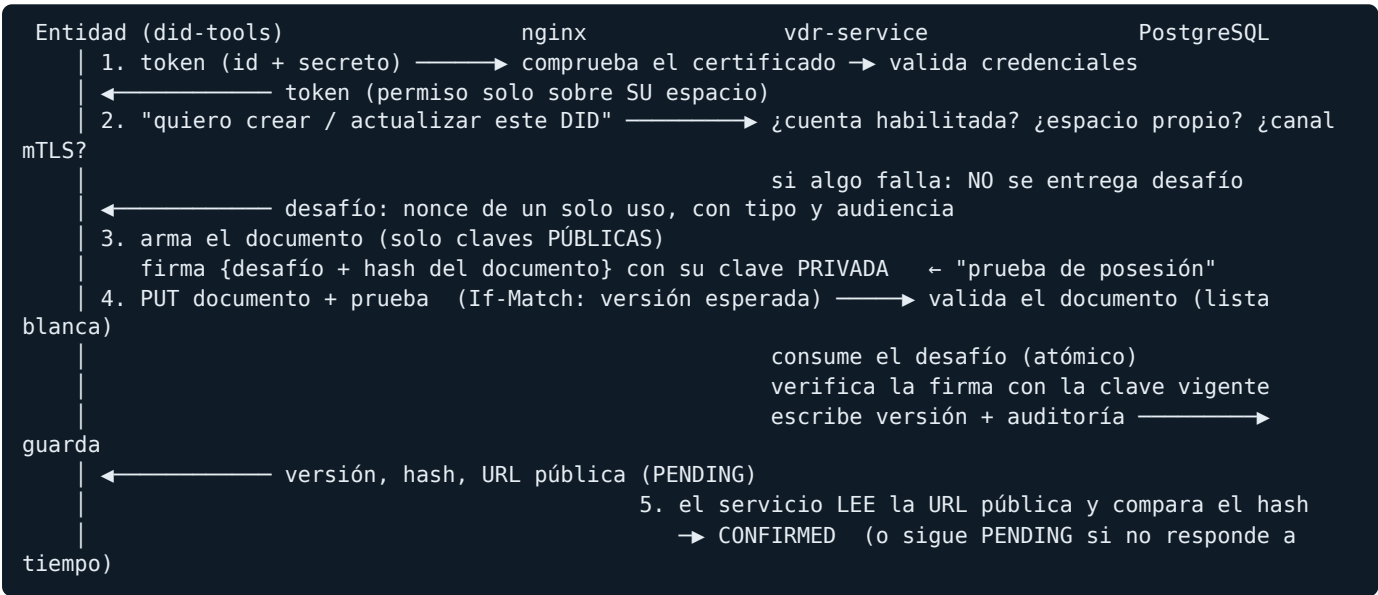
Tabla	Para qué
did_documents	La foto de hoy: una fila por DID, con su estado y su versión vigente
did_document_versions	El álbum: una fila por cada versión de cada DID. Solo se agrega
challenges	Los desafíos (valores de un solo uso) que se entregan antes de escribir
operations	Cada intento de escritura, con su estado (PENDING , CONFIRMED ...)
audit_log	El diario: quién hizo qué y cuándo, incluidos los rechazos . Solo se agrega
entity_accounts , namespaces	Las entidades autorizadas y el espacio de nombres de cada una
schema_migrations	Un control interno: qué versiones del esquema de la base ya se aplicaron

⚠ **No te confundas con lo que sobra.** En el listado verás además varias tablas que empiezan por `wallet_` y un *trigger* `wallet_audit_append_only`. Pertenecen al *Wallet Backend* de otras ERSO (001 a 003), que comparte esta misma base de datos. **No intervienen en la ERSO 004.** Las siete tablas de arriba (sin `schema_migrations`) son las del registro.

Los *triggers* `versions_append_only` y `audit_append_only` son la **regla inquebrantable**: cualquier intento de modificar o borrar filas de esas dos tablas falla, **aunque venga de alguien con acceso directo a la base**.

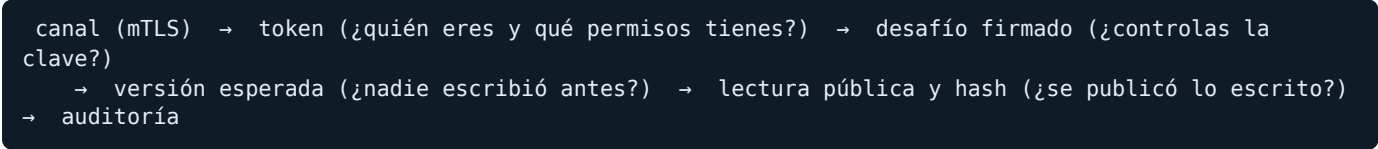
13. Cómo funciona por dentro (lo mínimo para entender las pruebas)

13.1 Qué ocurre cuando una entidad publica un documento



💡 **Frase clave:** "no se da por publicado hasta que se lee de vuelta desde la URL pública y el hash coincide".

13.2 Las capas de una escritura (idea madre 5)



Si un atacante roba **una** de esas cosas, le quedan las demás por superar. Es lo que se demuestra en el experimento 3 del criterio 1.

13.3 Mapa de requisitos a implementación

Requisito de la ERSO	Cómo se resolvió	Dónde (archivo)
Endpoints de registro	Rutas públicas y de administración	vdr-service/.../Server.kt

Requisito de la ERSO	Cómo se resolvió	Dónde (archivo)
Adaptador al método DID	DidMethodAdapter / DidWebAdapter	vdr-service/.../RegistryService.kt , did-core/.../DidWeb.kt
Registrar y actualizar	Tablas y lógica de escritura con versión esperada	V1__init.sql , RegistryService.write
Respalidar y recuperar	Exportar/importar con checksum y verificación	RegistryService.backup/restore
Camino base con la opción apagada	if (cfg.vdrEnabled) al arrancar; rutas condicionadas	Main.kt , Server.kt , AppConfig.kt
Aislamiento	Contenedores y solo nginx publicando	deploy/docker-compose.yml , nginx.conf

PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA CRITERIO

14. Cómo funcionan los laboratorios

 **Cómo funciona esta parte.** Cada laboratorio tiene siempre la misma forma: **(1)** el objetivo, **(2)** unas predicciones para que pienses *antes* de ver el resultado, **(3)** la sesión de terminal real con cada comando y la respuesta del sistema, **(4)** la explicación de qué hace cada comando y qué se busca, **(5)** la lectura del resultado con las respuestas a las predicciones y **(6)** la conclusión: la **evidencia** que aporta.

Mapa de laboratorios

Sección	Laboratorio	Qué demuestra	Script equivalente
§15	A — Capacidades	Aislamiento, opcional, adaptador y las dos puertas (pasos 1 y 2 del PDF)	scripts/lab-004-capacidades.sh
§16	B — Condiciones	Aislado sin afectar al resto, forma según el método DID, marca "O"	scripts/lab-004-condiciones.sh
§17	Criterio 1	Registro, actualización y trazabilidad	scripts/lab-004-registro.sh
§18	Criterio 2	Respaldo y recuperación verificados	scripts/lab-004-respaldo.sh
§19	Criterio 3	El camino base opera con la opción apagada	scripts/lab-004-camino-base.sh
§20	Pruebas automáticas y E2E	Las mismas verificaciones, repetibles por un programa	scripts/lab-004-pruebas.sh

 **Antes de empezar cualquier laboratorio**, el terminal debe estar en la carpeta `deploy` con las variables cargadas y los atajos definidos (sección 12.4 y 12.6). Si abres un terminal nuevo, repite esos comandos. Si al pegar comandos largos se corrompen caracteres, ejecuta el script equivalente de la tabla.

15. Laboratorio A — Capacidades: aislado, opcional, adaptador y dos puertas

Objetivo. Demostrar cuatro cosas que el PDF pide en sus pasos 1 y 2 y en la frase de capacidades: que el entorno es **aislado**, que la extensión es **opcional**, que hay un **adaptador** y que existen **dos puertas** distintas.

Predice antes de ejecutar

- **P1.** ¿Qué contenedores crees que tendrán puertos publicados hacia tu máquina?
- **P2.** Si intentas entrar directo a la aplicación (127.0.0.1:8080), saltándote nginx, ¿funciona?
- **P3.** Si lo intentas **desde dentro** de la red de Docker, ¿funciona? ¿Qué te dice eso sobre qué significa "aislado"?
- **P4.** Con la extensión apagada, ¿qué dirán /health y /base/ping? ¿Y el did.json de Avance?
- **P5.** Un PUT por la puerta de lectura, ¿qué código da? ¿Y consultar /admin/v1/... por esa misma puerta?

15.1 ¿Qué puertos están abiertos hacia el exterior?

```
$ docker compose ps --format 'table {{.Service}}\t{{.Ports}}' | sed 's/\[:\]\:[0-9]*->[0-9]*\tcp,\?\n?//g'
SERVICE      PORTS
credential
nginx          80/tcp, 0.0.0.0:8444->8444/tcp, 0.0.0.0:8443->443/tcp, 0.0.0.0:9443->8443/tcp,
postgres       5432/tcp
vdr
wallet
$ ss -ltn | grep -E ":(8080|8081|5432|8090|8100) " || echo "ninguno: 8080, 8081, 5432, 8090 y 8100 NO
están publicados"
ninguno: 8080, 8081, 5432, 8090 y 8100 NO están publicados
```

Comando	Qué hace	Qué se busca
<code>docker compose ps --format 'table ...'</code>	Lista los servicios y sus puertos. El <code>sed</code> solo quita ruido (direcciones IPv6)	Ver quién publica puertos
<code>ss -ltn grep -E ":(8080 8081 5432 8090 8100) "</code>	<code>ss</code> lista los puertos que escuchan en tu máquina ; <code>grep</code> busca los internos	Comprobar que la aplicación y la base no escuchan hacia fuera

✓ **Lectura.** Solo `nginx` tiene flechas `0.0.0.0:8080->8080`. `postgres` dice `5432/tcp` **sin flecha**: abierto dentro del contenedor, **no publicado**. La segunda consulta imprime el mensaje de que **ninguno** de los puertos internos está publicado. **Respuesta a P1:** solo `nginx`.

15.2 Entrar "por la puerta de atrás" frente a entrar desde dentro

```
$ curl -s -m 3 http://127.0.0.1:8080/health; echo "código de salida de curl: $?"
código de salida de curl: 7
$ docker compose exec -T nginx wget -qO- http://vdr:8080/health; echo
{"status":"UP","vdr":"enabled"}
```

Comando	Qué hace	Qué se busca
<code>curl -s -m 3 http://127.0.0.1:8080/health</code>	Intenta hablar directo con la aplicación, sin pasar por nginx. <code>-m 3</code> : máximo 3 segundos	Que no se pueda
<code>echo "... \$?"</code>	Muestra el <i>código de salida</i> de <code>curl</code> (7 significa "no pude conectar")	Confirmarlo

Comando	Qué hace	Qué se busca
<code>docker compose exec -T nginx wget -q0- http://vdr:8080/health</code>	Ejecuta <code>wget</code> dentro del contenedor de nginx , que sí está en la red privada, y consulta a la aplicación por su nombre interno <code>vdr</code>	Que desde dentro sí se pueda

✅ **Lectura. Respuestas a P2 y P3:** desde fuera no hay conexión (código de salida 7); desde dentro de la red de Docker responde `{"status":"UP","vdr":"enabled"}`. Las dos cosas son ciertas a la vez porque **depende de dónde preguntes**. 💡 *Aislado no es inexistente*: la aplicación está viva, pero sin puerta hacia el exterior.

15.3 La extensión opcional: una segunda copia, apagada

```
$ docker run -d --rm --name vdr-off -e VDR_ENABLED=false -p 18080:8080 vdr-ssi/vdr-service:local
>/dev/null; sleep 6
$ curl -s http://127.0.0.1:18080/health
{"status":"UP","vdr":"disabled"}
$ curl -s http://127.0.0.1:18080/base/ping
pong
$ curl -s -o /dev/null -w "HTTP %{http_code}\n" http://127.0.0.1:18080/entidades/avance/did.json
HTTP 404
$ docker logs vdr-off 2>&1 | grep "VDR APAGADO" | sed "s/^[0-9:. ]*//"
INFO vdr.main - VDR APAGADO (VDR_ENABLED=false): solo el camino base está activo
$ docker rm -f vdr-off
vdr-off
```

Comando	Qué hace	Qué se busca
<code>docker run -d --rm --name vdr-off -e VDR_ENABLED=false -p 18080:8080 ...</code>	Arranca una segunda copia del servicio, temporal, con la extensión apagada (<code>-e VDR_ENABLED=false</code>) y publicada en el puerto 18080	Observar el comportamiento apagado sin tocar el proyecto
<code>curl ... /health y /base/ping</code>	Consulta el camino base	Que sigan respondiendo
<code>curl ... /entidades/avance/did.json</code>	Pide un documento del registro	Que no exista esa ruta
<code>docker logs vdr-off grep "VDR APAGADO"</code>	Lee lo que el servicio dijo al arrancar	La confirmación escrita
<code>docker rm -f vdr-off</code>	Borra la copia temporal	Dejar todo limpio

✅ **Lectura. Respuesta a P4:** `/health` dice `"vdr":"disabled"` (sigue **UP**), `/base/ping` responde `pong`, el `did.json` da **404** porque esa ruta **ni siquiera se registra** cuando la extensión está apagada. El registro lo confirma: `"VDR APAGADO (VDR_ENABLED=false): solo el camino base está activo"`.

15.4 El adaptador: cómo un DID se convierte en la "forma" del registro

```
$ docker compose --progress quiet --profile tools run --rm -T tools did --domain $D --namespace
entidades/avance
did:web:civica-desarrollo.avance.org.co:entidades:avance
https://civica-desarrollo.avance.org.co/entidades/avance/did.json
```

Comando	Qué hace	Qué se busca
<code>tools did --domain ... --namespace entidades/avance</code>	Pide a la herramienta que calcule el DID de una entidad y la dirección donde vive su documento	Ver la regla <code>did:web</code> en acción

✓ **Lectura.** La primera línea es el DID. La segunda, su **dirección web**: los `:` después del dominio pasaron a `/` y se añadió `did.json`. Esa traducción es el trabajo del **adaptador**.

15.5 Las dos puertas

```
$ curl -s -o /dev/null -w "GET did.json -> HTTP %{http_code}\n" $PUB
https://$D:8443/entidades/avance/did.json
GET did.json -> HTTP 200
$ curl -s -o /dev/null -w "PUT did.json -> HTTP %{http_code}\n" -X PUT $PUB
https://$D:8443/entidades/avance/did.json
PUT did.json -> HTTP 403
$ curl -s -o /dev/null -w "GET /admin/v1/audit -> HTTP %{http_code}\n" $PUB
https://$D:8443/admin/v1/audit
GET /admin/v1/audit -> HTTP 404
$ curl -s -o /dev/null -w "escritura sin certificado -> HTTP %{http_code}\n" $ADM
https://$D:9443/admin/v1/audit
escritura sin certificado -> HTTP 400
```

Comando	Qué hace	Qué se busca
<code>GET did.json</code> por el puerto 8443	Leer por la puerta pública	Debe funcionar
<code>PUT did.json</code> por el puerto 8443	Intentar escribir por la puerta pública	Debe rechazarse
<code>GET /admin/v1/audit</code> por el puerto 8443	Intentar usar la administración por la puerta pública	No debe existir ahí
<code>...:9443/admin/v1/audit</code> sin certificado	Intentar entrar por la puerta de escritura sin certificado cliente	nginx debe cortar

✓ **Lectura. Respuesta a P5:** 200 leer; 403 el PUT está **prohibido** por la puerta pública; 404 la administración **no existe** por esa puerta; 400 en la puerta de escritura sin certificado: **nginx** la corta antes de llegar a la aplicación.

🎯 Conclusión del laboratorio A

Lo que pide el PDF	Evidencia
Entorno aislado (paso 1)	Solo nginx publica; la aplicación y la base no se alcanzan desde fuera (§15.1 y §15.2)
Extensión opcional	Con <code>VDR_ENABLED=false</code> solo corre el camino base (§15.3)
Adaptador (paso 2)	El DID se traduce a su dirección (§15.4)
Endpoints (capacidad 1)	Dos puertas separadas con reglas distintas (§15.5)

16. Laboratorio B — Condiciones: aislado, según el método y marca "O"

Objetivo. Comprobar las tres condiciones del PDF con observaciones concretas.

Predice antes de ejecutar

- **P1.** Si detienes el contenedor del VDR, ¿qué código da el `did.json`? ¿Y la cartera y el emisor de credenciales (que no son parte de esta ERSO)?
- **P2.** Si pides resolver un DID de **otro método** (`did:ebssi:...`), ¿qué responde el sistema?
- **P3.** De estas cinco configuraciones, ¿en cuáles arranca **encendido**? (*sin variable* · `false` · `si` · `True` · `true` *sin secreto*)

16.1 Condición 1 — Aislado: ¿qué pasa si el VDR se cae?

```
$ curl -s -o /dev/null -w "VDR did.json de avance      -> HTTP %{http_code}\n" $PUB
https://$D:8443/entidades/avance/did.json
VDR did.json de avance      -> HTTP 200
$ curl -s -o /dev/null -w "Wallet (instancia inexistente) -> HTTP %{http_code}\n" $CRED
https://$D:8444/wallet/v1/instances/00000000-0000-0000-0000-000000000000
Wallet (instancia inexistente) -> HTTP 401
$ curl -s -o /dev/null -w "Credenciales (metadatos)      -> HTTP %{http_code}\n" $CRED
https://$D:8444/.well-known/openid-credential-issuer
Credenciales (metadatos)      -> HTTP 200
```

Todo está en marcha. Ahora **detenemos el contenedor del VDR** (como si fallara) y volvemos a preguntar:

```
$ docker compose stop vdr
Container vdr-ssi-vdr-1 Stopping
Container vdr-ssi-vdr-1 Stopped
$ curl -s -o /dev/null -w "VDR did.json de avance      -> HTTP %{http_code}\n" $PUB
https://$D:8443/entidades/avance/did.json
VDR did.json de avance      -> HTTP 502
$ curl -s -o /dev/null -w "Wallet (instancia inexistente) -> HTTP %{http_code}\n" $CRED
https://$D:8444/wallet/v1/instances/00000000-0000-0000-0000-000000000000
Wallet (instancia inexistente) -> HTTP 401
$ curl -s -o /dev/null -w "Credenciales (metadatos)      -> HTTP %{http_code}\n" $CRED
https://$D:8444/.well-known/openid-credential-issuer
Credenciales (metadatos)      -> HTTP 200
$ docker compose ps --format '{{.Service}} {{.State}}'
credential running
nginx running
postgres running
wallet running
```

Lo volvemos a encender y comprobamos que se recupera:

```
$ docker compose start vdr
Container vdr-ssi-postgres-1 Waiting
Container vdr-ssi-postgres-1 Healthy
Container vdr-ssi-vdr-1 Starting
Container vdr-ssi-vdr-1 Started
$ for i in $(seq 1 40); do curl -sf $PUB https://$D:8443/health >/dev/null && break; sleep 1; done
$ curl -s -o /dev/null -w "VDR did.json de avance      -> HTTP %{http_code}\n" $PUB
https://$D:8443/entidades/avance/did.json
VDR did.json de avance      -> HTTP 200
```

Comando	Qué hace	Qué se busca
<code>docker compose stop vdr</code>	Detiene solo el contenedor del VDR	Simular un fallo de la extensión
<code>curl ... -w "HTTP %{http_code}"</code>	Imprime solo el código de respuesta	Ver quién responde
<code>docker compose ps --format '{{.Service}} {{.State}}'</code>	Lista el estado de cada servicio	Ver que los demás siguen <code>running</code>
<code>docker compose start vdr + espera</code>	Lo vuelve a encender	Comprobar la recuperación

✓ **Lectura. Respuesta a P1:** con el VDR detenido el `did.json` da **502** ("puerta de enlace incorrecta": **nginx** está vivo pero no encuentra al servicio que debería atender). La cartera (401) y el emisor (200) **siguen respondiendo igual**. 💡 El fallo quedó contenido en el VDR. El 502 es la señal **correcta** de aislamiento: lo que cayó fue la extensión, no el sistema.

16.2 Condición 2 — La forma depende del método DID

```
$ grep -n -A8 "interface DidMethodAdapter" ../vdr-
service/src/main/kotlin/co/org/avance/ssi/vdr/RegistryService.kt | cut -c1-120
37:interface DidMethodAdapter {
38-     val method: String
39-     fun parse(did: String): DidWebId
40-     fun namespaceOf(did: String): String
41-     fun publicPath(did: String): String
42-     fun didFromUrlPath(host: String, segments: List<String>): String?
43-     fun didForNamespace(host: String, namespace: String): String
44-}
45-
```

```
$ docker compose --progress quiet --profile tools run --rm -T tools resolve --did
did:ebssi:z2FPyQqulzrZvwFdtHHdsFf1 --ca /certs/ca.crt 2>&1 | grep -i "error" | cut -c1-200
resolutionMetadata: ResolutionMetadata(error=invalidDid, message=El DID debe usar el método did:web,
contentType=null, url=null, violations=[])
```

Comando	Qué hace	Qué se busca
<code>grep -n -A8 "interface DidMethodAdapter" ...</code>	Muestra en el código la interfaz del adaptador	Ver qué "traduce"
<code>tools resolve --did did:ebssi:...</code>	Pide resolver un DID de otro método	Que sin adaptador no se acepte

✓ **Lectura. Respuesta a P2:** el sistema responde `error=invalidDid` con el mensaje "El DID debe usar el método `did:web`". Y mira la interfaz: `fun parse(did: String): DidWebId` devuelve un tipo llamado `DidWebId`. ⚠ **Precisión honesta:** el adaptador es el lugar correcto para aislar el método, pero hoy esa abstracción **todavía tiene forma de `did:web`**; para agregar otro método habría que generalizar también ese tipo.

16.3 Condición 3 — Marca "O": desactivada por defecto

```
$ docker run -d --name vdr-cond vdr-ssi/vdr-service:local >/dev/null; sleep 5; docker logs vdr-cond
2>&1 | grep -E "VDR APAGADO|VDR HABILITADO" | sed "s/^[0-9:. ]*//"; docker rm -f vdr-cond >/dev/null
INFO vdr.main - VDR APAGADO (VDR_ENABLED=false): solo el camino base está activo
$ docker run -d --name vdr-cond -e VDR_ENABLED=si vdr-ssi/vdr-service:local >/dev/null; sleep 5;
docker logs vdr-cond 2>&1 | grep -E "VDR APAGADO|VDR HABILITADO" | sed "s/^[0-9:. ]*//"; docker rm -f
```

```
vdr-cond >/dev/null
INFO vdr.main - VDR APAGADO (VDR_ENABLED=false): solo el camino base está activo
$ docker run -d --name vdr-cond -e VDR_ENABLED=true vdr-ssi/vdr-service:local >/dev/null; sleep 5;
docker logs vdr-cond 2>&1 | grep -E "debe tener" | sed "s/^.*IllegalArgumentException: //" ; docker ps
-a --filter name=vdr-cond --format "estado del contenedor: {{.Status}}"; docker rm -f vdr-cond
>/dev/null
VDR_JWT_SECRET debe tener al menos 32 caracteres cuando VDR_ENABLED=true
estado del contenedor: Exited (1) 4 seconds ago
```

Comando	Qué hace	Qué se busca
<code>docker run -d --name vdr-cond ...</code> (sin variable)	Arranca una copia sin definir VDR_ENABLED	Ver el comportamiento por defecto
<code>con -e VDR_ENABLED=si</code>	Un valor no válido	Que no encienda
<code>con -e VDR_ENABLED=true</code> (sin secreto)	Intenta encender sin la configuración de seguridad	Que se niegue a arrancar
<code>docker logs ... grep ...</code>	Lee el mensaje de arranque	La confirmación

✓ **Lectura. Respuesta a P3:** por defecto → **apagado**. si → **apagado** (solo `true` exacto enciende: *falla cerrada*; `True` y `1` también quedan apagados, como se comprueba en el laboratorio del criterio 3). `true` **sin secreto** → no arranca y dice "`VDR_JWT_SECRET debe tener al menos 32 caracteres cuando VDR_ENABLED=true`" (*falla temprana*). Así que, de las cinco configuraciones, **ninguna** arranca encendida salvo `true` **bien configurado**.

Preguntas de correlación (con respuesta)

1. El `502` lo da `nginx`, no la aplicación. ¿Por qué es la señal correcta de "aislado"? Porque demuestra que la puerta de entrada y los demás servicios siguen vivos: el fallo no se propagó.
2. ¿Para soportar otro método basta con escribir otro adaptador? **No del todo**: hay que escribirlo **y** generalizar el tipo `DidWebId` que la interfaz devuelve.
3. ¿Por qué es mejor que `true` sin secreto no arranque, en vez de arrancar y avisar? Porque un error visible en el **despliegue** es barato; un servicio inseguro funcionando puede pasar desapercibido hasta que alguien lo ataque.

17. CRITERIO 1 — El registro y la actualización de entradas son operativos

"El registro y la actualización de entradas son operativos; evidencia: pruebas de registro, actualización y trazabilidad."

Objetivo. Mostrar que se puede **crear** y **actualizar** un documento, que cada cambio queda **trazado**, y — lo más importante — que los intentos indebidos **fallan**.

Qué es lo que se va a demostrar (afirmación comprobable):

1. Un documento existente **no puede volver a crearse**.
2. Una **actualización legítima** (con la versión esperada y la firma de la clave vigente) crea la versión siguiente y el documento público cambia.

3. El hash del documento **público** es el de la versión **escrita**.
4. Un atacante con **certificado y token** pero **sin la clave privada** no puede cambiarlo.
5. Una escritura con una **versión desactualizada** se rechaza.
6. Todo queda en el historial y la auditoría, y **no se puede borrar**.

Predice antes de ejecutar

- **P1.** Para **actualizar** un DID que está en la versión 1, ¿qué valor de `If-Match` envías? ¿Qué pasaría si enviaras `0`?
- **P2.** ¿Con qué clave se firma una actualización: la **nueva** o la **vigente**? ¿Qué sería peligroso en el otro caso?
- **P3.** Después de la actualización, ¿cuántas filas habrá en `did_document_versions` para este DID, y cuántas en `did_documents`?
- **P4.** El atacante tiene certificado, token y desafío. ¿Qué le falta?

17.1 Preparar el laboratorio

Actuamos como la entidad "**lab-operator**", dueña del espacio `lab/laboratorio`. Definimos tres atajos:

```
$ DID=did:web:$D:lab:laboratorio
$ TOOLS() { docker compose --progress quiet --profile tools run --rm -T tools "$@"; }
$ LABOP() { TOOLS admin --admin-url "https://$D:8443" --client-id lab-operator --secret "$CLIENT_SECRET_LAB" --p12 /certs/lab-operator.p12 --p12-pass "$P12_PASS" --ca /certs/ca.crt "$@"; }
$ WRITE() { TOOLS write --admin-url "https://$D:8443" --client-id lab-operator --secret "$CLIENT_SECRET_LAB" --p12 /certs/lab-operator.p12 --p12-pass "$P12_PASS" --ca /certs/ca.crt --did $DID "$@"; }
```

Atajo	Qué hace
DID	El identificador con el que vamos a trabajar
TOOLS	Ejecuta la herramienta <code>did-tools</code> dentro de un contenedor (con la carpeta de claves montada en <code>/work</code> y los certificados en <code>/certs</code>)
LABOP	Hace una consulta de administración como lab-operator : usa su identificador, su secreto y su certificado cliente (<code>.p12</code>), y confía en nuestra CA
WRITE	Hace una escritura como <code>lab-operator</code> sobre ese DID

17.2 Estado de partida

```
$ LABOP --path "/admin/v1/documents/$DID/versions" 2>&1 | sed -n '/^\[/,$p' | jq -r '.[ ] | "\v\ (.version) \(.operation) \(.hash[0:26])..."'
v1 CREATE sha256:cdb0013fb29544cd2c0...
v2 UPDATE sha256:4dfd728d7017283409c...
$ V=$(LABOP --path "/admin/v1/documents/$DID/versions" 2>&1 | sed -n '/^\[/,$p' | jq 'length'); echo "versión actual = $V"
versión actual = 2
```

Comando	Qué hace	Qué se busca
<code>LABOP --path "/admin/v1/documents/\$DID/versions"</code>	Pide al registro la lista de versiones de ese DID (canal de escritura, autenticado)	Saber desde qué versión partimos

Comando	Qué hace	Qué se busca
jq ...	jq da formato a la respuesta JSON	Leerla fácil
V=\$(... jq 'length')	Guarda en la variable V cuántas versiones hay	Usarla como "versión esperada"

✓ **Lectura.** Muestra la versión actual de partida y la guarda en V.

17.3 Experimento 1 — Volver a crear un DID que ya existe

```
$ TTOOLS build-doc --did $DID --key lab.json --out lab-doc-base.json >/dev/null
$ WRITE --purpose CREATE --expected 0 --key lab.json --doc lab-doc-base.json 2>&1 | tail -3
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 409 {"error":"ALREADY_EXISTS","message":"El DID ya existe (versión 2); use UPDATE"}
```

Comando	Qué hace	Qué se busca
TTOOLS build-doc --did ... --key lab.json --out ...	Arma un DID Document con la clave pública de la entidad (el archivo lab.json guarda su par de claves)	Tener un documento válido que enviar
WRITE --purpose CREATE --expected 0 ...	Intenta crear (--purpose CREATE) diciendo "espero que no exista" (--expected 0 , que se envía como If-Match: 0)	Que el registro lo rechace porque ya existe

✓ **Lectura.** El registro responde **409 ALREADY_EXISTS** ("El DID ya existe (versión N); use UPDATE"). Nótese que ni siquiera entregó un desafío: rechazó **antes** de gastar nada. **Respuesta a P1 (parte 2):** con 0 se obtiene este rechazo.

17.4 Experimento 2 — Actualización legítima

```
$ TTOOLS build-doc --did $DID --key lab.json --service-url "https://$D/lab/servicio-$(date +%s)" --out lab-doc-nuevo.json >/dev/null
$ WRITE --purpose UPDATE --expected $V --key lab.json --doc lab-doc-nuevo.json 2>&1
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=dce772cf-6dc0-42c2-b40f-27a949cd814c audience=vdr:civica-desarrollo.avance.org.co:did-operation purpose=UPDATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:lab:laboratorio#key-1 docHash=sha256:01816778980b434e34a551e99c6a2e228b94726d29412e617fa2a76b3e99b4b1
[4/4] escritura .... 200 If-Match=2 Idempotency-Key=3d649c77-d217-4b13-8a92-3127ea63f32b
{
  "operationId": "16e3e529-963b-44c9-96c8-f98002d097f7",
  "did": "did:web:civica-desarrollo.avance.org.co:lab:laboratorio",
  "purpose": "UPDATE",
  "status": "CONFIRMED",
  "version": 3,
  "hash": "sha256:01816778980b434e34a551e99c6a2e228b94726d29412e617fa2a76b3e99b4b1",
  "publicUrl": "https://civica-desarrollo.avance.org.co/lab/laboratorio/did.json"
}
```

Parte de la salida	Qué significa
[1/4] token ... OK	Se obtuvo un token (identificador + secreto) por el canal con certificado cliente
[2/4] desafío ... 201 ...	El servidor entregó un desafío de un solo uso, con tipo (UPDATE) y audiencia

Parte de la salida	Qué significa
[3/4] prueba ... ES256 kid=... docHash=...	La entidad firmó el desafío y el hash del documento con su clave privada
[4/4] escritura ... 200 If-Match=N	Se escribió indicando la versión que esperaba (If-Match: N)
El JSON final	status: CONFIRMED , la version nueva, el hash y la publicUrl

✓ **Lectura. Respuestas a P1 y P2.** Para actualizar un DID en la versión **N** se envía If-Match: N ("actualiza solo si la versión actual sigue siendo N"). La firma es con la **clave vigente**: si bastara la clave *nueva*, cualquiera podría proponer un documento con su propia clave y adueñarse del DID. **CONFIRMED** significa además que el servidor **leyó de vuelta la URL pública** y el hash coincidió.

17.5 ¿Lo publicado es lo escrito?

```
$ curl -s $PUB https://$D:8443/lab/laboratorio/did.json | sha256sum | awk '{print "hash del documento público : sha256:$1}'
hash del documento público :
sha256:01816778980b434e34a551e99c6a2e228b94726d29412e617fa2a76b3e99b4b1
$ LABOP --path "/admin/v1/documents/$DID/versions" 2>&1 | sed -n '/^\[/,$p' | jq -r '"hash de la última versión : " + .[-1].hash'
hash de la última versión : sha256:01816778980b434e34a551e99c6a2e228b94726d29412e617fa2a76b3e99b4b1
```

Comando	Qué hace	Qué se busca
curl ... lab/laboratorio/did.json sha256sum	Descarga el documento público y calcula su huella con sha256sum	La huella de lo que ve el mundo
LABOP ... versions jq [-1].hash	Pide el hash de la última versión registrada	La huella que el registro dice haber escrito

✓ **Lectura.** Las dos huellas son **idénticas**. Eso es lo que significa **CONFIRMED**: *lo que cualquiera lee es exactamente lo que se escribió*.

17.6 Experimento 3 — El atacante con certificado y token, pero sin la clave

```
$ T00LS keygen --out lab-intruso.json
Clave P-256 generada. Multikey público:zDnaeqUeRrHwcRwsC34HEXuNEbTrRHt8H1qDoBPrW1lzebRvA
$ T00LS build-doc --did $DID --key lab-intruso.json --service-url "https://sitio-malicioso.example" -
-out lab-doc-malo.json >/dev/null
$ WRITE --purpose UPDATE --expected $((V+1)) --key lab-intruso.json --doc lab-doc-malo.json 2>&1 |
tail -6
[4/4] escritura .... 403 If-Match=3 Idempotency-Key=9a0cfd55-a872-46de-a96a-fd6702d11b16
{
  "error": "INVALID_PROOF",
  "message": "Prueba de posesión inválida: firma no válida"
}
```

Comando	Qué hace	Qué se busca
T00LS keygen --out lab-intruso.json	Genera un par de claves nuevo : las del atacante	Simular a quien no tiene la clave del DID

Comando	Qué hace	Qué se busca
T00LS build-doc ... --key lab-intruso.json --service-url https://sitio-malicioso.example	Arma un documento con la clave del atacante y apuntando a su sitio	El documento "malicioso"
WRITE ... --key lab-intruso.json	Intenta publicarlo firmando con la clave del atacante (aunque entra con el certificado, el token y el desafío legítimos de lab-operator)	Que el registro lo rechace

✓ **Lectura. Respuesta a P4.** El registro responde `403 INVALID_PROOF` — "*Prueba de posesión inválida: firma no válida*". El atacante tenía **tres de cuatro cosas** (certificado, token y desafío); le faltó **la clave privada vigente** del DID. 💡 *La pública verifica, la privada firma*: la clave pública del DID está a la vista de todos, así que nunca es lo que "falta".

17.7 Experimento 4 — Versión desactualizada

```
$ WRITE --purpose UPDATE --expected $V --key lab.json --doc lab-doc-nuevo.json 2>&1
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=392e764d-bfa1-47bf-8e5d-a2f06dda6357 audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=UPDATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:lab:laboratorio#key-1
docHash=sha256:01816778980b434e34a551e99c6a2e228b94726d29412e617fa2a76b3e99b4b1
[4/4] escritura .... 412 If-Match=2 Idempotency-Key=2b4fb2f5-c874-4872-a6fd-f9951eda234c
{
  "error": "VERSION_CONFLICT",
  "message": "Versión esperada 2, versión actual 3",
  "details": [
    "currentVersion=3"
  ]
}
```

✓ **Lectura.** `412 VERSION_CONFLICT` con el mensaje "*Versión esperada N, versión actual N+1*" y `currentVersion`. Es la **concurrency optimista**: si dos personas editan a la vez, la segunda recibe un aviso en lugar de pisar silenciosamente el trabajo de la primera.

17.8 Trazabilidad: el historial y la auditoría

```
$ LABOP --path "/admin/v1/documents/$DID/versions" 2>&1 | sed -n '/^[/,,$p' | jq -r '.[ ] | "v\
(.version) \(.operation) actor=\(.actor) \(.createdAt) \(.hash[0:26])..."'
v1 CREATE actor=lab-operator 2026-09-30T14:20:53.882318Z sha256:cdb0013fb29544cd2c0...
v2 UPDATE actor=lab-operator 2026-10-01T14:56:46.340823Z sha256:4dfd728d7017283409c...
v3 UPDATE actor=lab-operator 2026-10-01T15:07:28.922634Z sha256:01816778980b434e34a...
$ LABOP --path "/admin/v1/audit?did=$DID" 2>&1 | sed -n '/^[/,,$p' | jq -r '.[ -9:][] | "\(.at[11:23])
\(.actor) \(.action) v\(.version // "-")"'
14:56:55.102 lab-operator CHALLENGE_ISSUED v-
14:56:55.127 lab-operator VERSION_CONFLICT v2
15:07:28.879 lab-operator CHALLENGE_ISSUED v-
15:07:28.922 lab-operator WRITE_UPDATE v3
15:07:29.070 system PUBLICATION_CONFIRMED v3
15:07:35.673 lab-operator CHALLENGE_ISSUED v-
15:07:35.696 lab-operator PROOF_REJECTED v3
15:07:37.701 lab-operator CHALLENGE_ISSUED v-
15:07:37.723 lab-operator VERSION_CONFLICT v3
```

Comando	Qué hace	Qué se busca
LABOP ... versions	Lista todas las versiones: número, operación, quién, cuándo y huella	El "álbum"
LABOP ... /audit? did=\$DID	Lista los eventos de auditoría de ese DID (.[-9:] = los últimos nueve)	El "diario"

✓ **Lectura.** El historial muestra CREATE y UPDATE con su actor y su hora. **La auditoría registra también lo que falló:** CHALLENGE_ISSUED → WRITE_UPDATE → PUBLICATION_CONFIRMED (lo bueno) y PROOF_REJECTED, VERSION_CONFLICT (los intentos rechazados). *Quien intenta romperlo deja huella.*

17.9 Inalterabilidad: intentar borrar o modificar el historial

```
$ $PSQL -c "delete from did_document_versions" 2>&1 | head -1
ERROR:  tabla append-only: DELETE no permitido sobre did_document_versions
$ $PSQL -c "update did_document_versions set hash='x'" 2>&1 | head -1
ERROR:  tabla append-only: UPDATE no permitido sobre did_document_versions
```

Comando	Qué hace	Qué se busca
\$PSQL -c "delete from did_document_versions"	Intenta borrar todo el historial directo en la base de datos	Que falle
\$PSQL -c "update ... set hash='x'"	Intenta modificar un hash	Que falle

✓ **Lectura. Respuesta a P3 (y a "¿por qué en la base y no en el código?").** Ambos intentos fallan con ERROR: tabla append-only: ... no permitido. La protección es un *trigger* **dentro de PostgreSQL:** funciona aunque haya un error de programación en la aplicación o alguien tenga acceso directo a la base. En cuanto a las filas: did_documents sigue teniendo **una** fila por DID (la foto) y did_document_versions una por **cada** versión (el álbum).

🎯 Conclusión — evidencia del criterio 1

Verbo del criterio	Evidencia en este laboratorio
Registro operativo	CREATE previo y 409 al repetirlo (§17.3)
Actualización operativa	UPDATE → 200 CONFIRMED, versión N+1, hash público = hash escrito (§17.4 y §17.5)
Trazabilidad	Historial con actor, hora y hash; auditoría con éxitos y rechazos (§17.8)
Caso negativo: clave ajena	403 INVALID_PROOF (§17.6)
Caso negativo: versión vieja	412 VERSION_CONFLICT (§17.7)
Caso negativo: reescribir la historia	append-only (§17.9)
Prueba automática equivalente	Erso004RegistryTest > criterio 1 - registro, actualizacion y trazabilidad son operativos (§20)

18. CRITERIO 2 — El respaldo y la recuperación se verifican

"El respaldo y la recuperación del registro se verifican; evidencia: prueba de respaldo y restauración."

Objetivo. Hacer un respaldo, **destruir** el estado del registro, intentar restaurar un respaldo **alterado** (debe fallar), restaurar el bueno y **comparar** que todo quedó idéntico.

💡 **Por qué "se verifican".** Un respaldo que nunca se restauró es una esperanza, no una garantía. El criterio exige **restaurar y comprobar**.

⚠️ **Este laboratorio es destructivo pero recuperable:** vacía las tablas del registro y las restaura desde el respaldo. Al terminar, el estado es idéntico (se demuestra en el paso 7).

🔧 Predice antes de ejecutar

- **P1.** Restaurar **sobre un registro con datos**: ¿se restaura o se rechaza? ¿Por qué se diseñó así?
- **P2.** Tras vaciar las tablas, ¿qué código da el `did.json` de Avance? ¿El servicio sigue vivo?
- **P3.** Si se cambia **un hash** dentro del archivo de respaldo, ¿qué dirá el servidor? Y tras el intento fallido, ¿cuántos DID habrá en la base: 0, parcialmente restaurados o todos?
- **P4.** Tras restaurar el bueno, ¿las huellas públicas serán **iguales** a las de antes? ¿Y el DID desactivado seguirá dando 410?
- **P5.** ¿Por qué el respaldo incluye `audit_log` y el historial, y no solo los documentos actuales?

18.1 Preparar atajos

```
$ ADMIN() { TOOLS admin --admin-url "https://$D:8443" --client-id vdr-admin --secret "$CLIENT_SECRET_ADMIN" --p12 /certs/vdr-admin.p12 --p12-pass "$P12_PASS" --ca /certs/ca.crt "$@"; }
$ HUELLA() { curl -s $PUB "https://$D:8443/$1/did.json" | sha256sum | cut -c1-16; }
$ CODIGO() { curl -s -o /dev/null -w "%{http_code}" $PUB "https://$D:8443/$1/did.json"; }
```

Atajo	Qué hace
ADMIN	Ejecuta una consulta de administración como <code>vdr-admin</code> , la única cuenta autorizada para respaldar y restaurar
HUELLA	Descarga el <code>did.json</code> público de un DID y calcula los 16 primeros caracteres de su hash
CODIGO	Devuelve solo el código HTTP de ese <code>did.json</code>

18.2 Paso 1 — La foto de "antes"

```
$ for p in entidades/avance lab/laboratorio; do echo "$p huella=$(HUELLA $p)"; done; echo
"entidades/avance-ciclo (desactivado) HTTP $(CODIGO entidades/avance-ciclo)"
entidades/avance huella=2ce8444ae71f931f
lab/laboratorio huella=01816778980b434e
entidades/avance-ciclo (desactivado) HTTP 410
$ $PSQL -tAc "select 'did_documents='||(select count(*) from did_documents)||' versiones='||(select
count(*) from did_document_versions)||' auditoria='||(select count(*) from audit_log)"
did_documents=4 versiones=9 auditoria=103
```

✓ **Lectura.** Quedan anotadas las **huellas** de dos documentos, el código 410 del DID **desactivado** y el conteo de filas (`did_documents`, `versiones`, `auditoria`). Es lo que compararemos al final.

18.3 Paso 2 — Hacer el respaldo

```
$ ADMIN --method POST --path /admin/v1/backup --out leccion-backup.json | tail -1
HTTP 200 -> guardado en leccion-backup.json (44868 bytes)
$ jq '{format, checksum, tablas: (.tables | to_entries | map({(.key): (.value|length)})) | add}}'
$W/leccion-backup.json
{
  "format": "vdr-backup/1",
  "checksum": "sha256:e75f4ee00e38ab613456e7cc9d50f6c260f99c435d905728e56bcbe986e87367",
  "tablas": {
    "entity_accounts": 4,
    "namespaces": 5,
    "did_documents": 4,
    "did_document_versions": 9,
    "operations": 9,
    "audit_log": 105
  }
}
```

Comando	Qué hace	Qué se busca
<code>ADMIN --method POST --path /admin/v1/backup --out leccion-backup.json</code>	Pide al registro su respaldo completo y lo guarda en un archivo	Tener la copia
<code>jq '{format, checksum, tablas: ...}'</code>	Resume el archivo: formato, checksum y cuántas filas trae cada tabla	Entender qué contiene

✓ **Lectura.** HTTP 200 y el archivo guardado. El resumen muestra `"format": "vdr-backup/1"` (la versión del formato), un `checksum: sha256:...` (la huella del **contenido completo**) y las **seis tablas**: cuentas, espacios de nombres, documentos, **versiones**, operaciones y **auditoría**. Por eso se respalda también el historial.

18.4 Paso 3 — Restaurar sobre un registro con datos

```
$ ADMIN --method POST --path /admin/v1/restore --body leccion-backup.json 2>&1 | head -4
HTTP 409
{
  "error": "REGISTRY_NOT_EMPTY",
  "message": "La restauración solo se permite sobre un registro vacío"
```

✓ **Lectura. Respuesta a P1:** 409 REGISTRY_NOT_EMPTY — *"La restauración solo se permite sobre un registro vacío"*. Se diseñó así para **no mezclar historiales** ni duplicar o pisar datos vivos.

18.5 Paso 4 — La pérdida total

```
$ $PSQL -q -c "TRUNCATE audit_log, operations, challenges, did_document_versions, did_documents, namespaces, entity_accounts CASCADE"
$ echo "avance HTTP $(CODIGO entidades/avance) laboratorio HTTP $(CODIGO lab/laboratorio)"; $PSQL -tAc "select 'did_documents=|'(select count(*) from did_documents)"
avance HTTP 404 laboratorio HTTP 404
did_documents=0
```

Comando	Qué hace
<code>\$PSQL -q -c "TRUNCATE audit_log, operations, challenges, did_document_versions, did_documents, namespaces, entity_accounts CASCADE"</code>	Vacía todas las tablas del registro de golpe. Simula perderlo todo

✓ **Lectura. Respuesta a P2:** los `did.json` dan **404** (ya no existen) y `did_documents=0`. El servicio **sigue vivo**; lo que desapareció fueron los datos.

18.6 Paso 5 — Un respaldo alterado

```
$ jq '.tables.did_document_versions[0].hash = "sha256:00"' $W/leccion-backup.json > $W/leccion-backup-alterado.json
$ ADMIN --method POST --path /admin/v1/restore --body leccion-backup-alterado.json 2>&1 | head -4
HTTP 422
{
  "error": "CHECKSUM_MISMATCH",
  "message": "El checksum del respaldo no coincide: archivo alterado o corrupto"
}
$ $PSQL -tAc "select 'tras el intento fallido: did_documents='||(select count(*) from did_documents)"
tras el intento fallido: did_documents=0
```

Comando	Qué hace	Qué se busca
<code>jq '.tables.did_document_versions[0].hash = "sha256:00"' ...</code>	Crea una copia del respaldo con un único hash cambiado	Simular corrupción o manipulación
<code>ADMIN ... restore --body leccion-backup-alterado.json</code>	Intenta restaurar esa copia	Que se rechace
consulta del conteo	Cuenta los DID tras el intento fallido	Comprobar que no quedó nada a medias

✓ **Lectura. Respuesta a P3:** `422 CHECKSUM_MISMATCH` — *"El checksum del respaldo no coincide: archivo alterado o corrupto"*. Y después `did_documents=0`: **nada quedó restaurado a medias**. Se explica por dos razones: el servidor **comprueba el checksum antes de escribir nada**, y la restauración ocurre dentro de una **transacción** (todo o nada).

18.7 Paso 6 — Restaurar el respaldo bueno

```
$ ADMIN --method POST --path /admin/v1/restore --body leccion-backup.json 2>&1 | head -14
HTTP 200
{
  "restored": {
    "entity_accounts": 4,
    "namespaces": 5,
    "did_documents": 4,
    "did_document_versions": 9,
    "operations": 9,
    "audit_log": 105
  },
  "documentsVerified": 4,
  "checksum": "sha256:e75f4ee00e38ab613456e7cc9d50f6c260f99c435d905728e56bcbe986e87367"
}
```

✓ **Lectura.** `HTTP 200` con el detalle de **cuántas filas se restauraron en cada tabla** y `"documentsVerified": N`. Ese campo es la **verificación interna**: tras restaurar, el servidor recalcula el hash de la versión vigente de **cada** DID y comprueba que coincide con el registrado.

18.8 Paso 7 — ¿Quedó idéntico?

```
$ for p in entidades/avance lab/laboratorio; do echo "$p huella=$(HUELLA $p)"; done; echo
"entidades/avance-ciclo (desactivado) HTTP $(CODIGO entidades/avance-ciclo)"
entidades/avance huella=2ce8444ae71f931f
lab/laboratorio huella=01816778980b434e
entidades/avance-ciclo (desactivado) HTTP 410
$ $PSQL -tAc "select 'did_documents='||(select count(*) from did_documents)||' versiones='||(select
count(*) from did_document_versions)||' auditoria='||(select count(*) from audit_log)"
did_documents=4 versiones=9 auditoria=108
$ $PSQL -tAc "select action, count(*) from audit_log where action in ('BACKUP','RESTORE') group by
action order by action"
BACKUP|4
RESTORE|4
```

✓ Lectura. Respuestas a P4 y P5.

- Las **huellas públicas son idénticas** a las del paso 1 y el DID desactivado sigue dando **410**: la restauración dejó el registro como estaba.
- Los conteos de documentos y versiones son los mismos. La **auditoría** puede tener algunas filas más que en el paso 1: son los eventos del propio laboratorio (el token del administrador, el respaldo y la restauración). 💡 *El historial de los respaldos y restauraciones anteriores también sobrevive*: por eso los contadores **BACKUP** y **RESTORE** suman todas las veces que se hizo.
- **P5**: el respaldo incluye el historial y la auditoría porque el criterio 1 exige **trazabilidad**: un registro restaurado sin ellos funcionaría, pero no podría probar quién hizo qué.

🔧 Preguntas de correlación (con respuesta)

1. *En el paso 5 no quedó nada a medias. ¿Qué concepto lo explica?* La **transacción** (todo o nada), junto con la validación del checksum **antes** de escribir.
2. *El criterio dice "se verifican". ¿Qué dos cosas del laboratorio constituyen esa verificación?*
(a) El **checksum** que rechaza un respaldo alterado (paso 5); (b) la **comparación posterior** de huellas, conteos y **documentsVerified** (pasos 6 y 7).

🎯 Conclusión — evidencia del criterio 2

Verbo del criterio	Evidencia
Respaldo	Archivo vdr-backup/1 con checksum y seis tablas (§18.3)
Recuperación	200 con filas restauradas (§18.7)
Se verifica	Huellas públicas idénticas, conteos iguales, documentsVerified , DID desactivado sigue en 410 (§18.8)
Negativo: datos vivos	409 REGISTRY_NOT_EMPTY (§18.4)
Negativo: archivo alterado	422 CHECKSUM_MISMATCH y nada a medias (§18.6)
Prueba automática equivalente	Erso004RegistryTest > criterio 2 - respaldo, perdida total y restauracion dejan el registro identico y solo el administrador puede respaldar y restaurar (§20)

19. CRITERIO 3 — El camino base opera sin el registro desplegado

"El camino base opera sin el registro desplegado; evidencia: pruebas de regresión con la opción apagada."

Objetivo. Demostrar que con la extensión **apagada** el camino base responde igual, las rutas del registro **no existen**, no se abre el canal de escritura y no se toca la base de datos. Y que **encendida**, el camino base no cambia.

 **Qué es una prueba de regresión.** Comprobar que *lo que ya funcionaba sigue funcionando* después de agregar algo nuevo.

Predice antes de ejecutar

- **P1.** Con la extensión apagada, ¿qué dirá `/health`? ¿`/base/ping`?
- **P2.** ¿Qué código dará el `did.json`: 404 de "no hay documento" o 404 de "no existe la ruta"? ¿Cómo lo distinguirías?
- **P3.** ¿Estará escuchando el canal de escritura (puerto 8081 del contenedor)?
- **P4.** ¿Se conectará a la base de datos? ¿Cómo lo comprobarías?
- **P5.** Con `VDR_ENABLED=si`, `True` o `1`, ¿se enciende?

19.1 Arrancar el servicio apagado, sin base de datos

```
$ docker rm -f vdr-off >/dev/null 2>&1; docker run -d --name vdr-off -p 18080:8080 vdr-ssi/vdr-service:local >/dev/null; sleep 6
$ docker logs vdr-off 2>&1 | sed 's/^[0-9:. ]*//' | grep -E 'VDR|Responding'
INFO vdr.main - VDR APAGADO (VDR_ENABLED=false): solo el camino base está activo
INFO i.k.s.Application - Responding at http://0.0.0.0:8080
```

Comando	Qué hace	Qué se busca
<code>docker run -d --name vdr-off -p 18080:8080 ...</code>	Arranca una copia temporal del servicio sin definir <code>VDR_ENABLED</code> y sin ninguna base de datos al lado	El comportamiento por defecto
<code>docker logs ... grep 'VDR Responding'</code>	Lee lo que dijo al arrancar	La confirmación

✓ **Lectura.** El servicio arrancó y dice "VDR APAGADO (`VDR_ENABLED=false`): solo el camino base está activo", y que está respondiendo en el puerto 8080. Arrancó **sin base de datos**, lo que ya demuestra que no la necesita apagado.

19.2 El camino base responde

```
$ curl -s http://127.0.0.1:18080/health
{"status":"UP","vdr":"disabled"}
$ curl -s http://127.0.0.1:18080/base/ping
pong
```

✓ **Lectura. Respuesta a P1:** `{"status":"UP","vdr":"disabled"}` (el servicio está **sano** y la extensión apagada) y `pong`.

19.3 Las rutas de la extensión no existen

```
$ curl -s -o /dev/null -w "GET /entidades/avance/did.json -> HTTP %{http_code}\n"
http://127.0.0.1:18080/entidades/avance/did.json
GET /entidades/avance/did.json -> HTTP 404
$ curl -s -o /dev/null -w "GET /.well-known/did.json -> HTTP %{http_code}\n"
http://127.0.0.1:18080/.well-known/did.json
GET /.well-known/did.json -> HTTP 404
$ IP=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' vdr-off); echo
"IP del contenedor: $IP"
IP del contenedor: 172.17.0.3
$ curl -s -m 3 -o /dev/null http://$IP:8081/admin/v1/challenges; echo "canal de escritura (puerto
8081): código de salida de curl = $"
canal de escritura (puerto 8081): código de salida de curl = 7
$ curl -s -m 3 -o /dev/null http://$IP:8080/health; echo "camino base (puerto 8080) : código
de salida de curl = $"
camino base (puerto 8080) : código de salida de curl = 0
```

Comando	Qué hace	Qué se busca
<code>curl ... /entidades/avance/did.json y /.well-known/did.json</code>	Pide documentos del registro	404
<code>IP=\$(docker inspect ...)</code>	Obtiene la dirección interna del contenedor temporal	Poder hablarle directamente
<code>curl http://\$IP:8081/...</code>	Intenta usar el canal de escritura (puerto 8081)	Que nadie escuche
<code>curl http://\$IP:8080/health</code>	Comprueba el camino base (puerto 8080)	Que sí se conecte

✓ **Lectura. Respuestas a P2 y P3:** los 404 son de "no existe la ruta": el servicio, apagado, **ni registra** esas rutas (no es que no haya documento). Y el puerto 8081 devuelve código de salida de curl = 7 (nadie escucha) mientras el 8080 da 0: **el canal de escritura ni siquiera se abre** cuando está apagado.

19.4 No tocó la base de datos

```
$ echo "líneas del registro que mencionan hikari, postgres o jdbc: $(docker logs vdr-off 2>&1 | grep
-ciE 'hikari|postgres|jdbc')"
líneas del registro que mencionan hikari, postgres o jdbc: 0
$ docker rm -f vdr-off
vdr-off
```

✓ **Lectura. Respuesta a P4:** el contador de líneas del registro que mencionan hikari, postgres o jdbc es 0. (Hikari es el componente que administra las conexiones a la base.) Apagado, el servicio no intenta conectarse.

19.5 Con la extensión encendida, el camino base no cambia

```
$ curl -s $PUB https://$D:8443/health
{"status":"UP","vdr":"enabled"}
$ curl -s $PUB https://$D:8443/base/ping
pong
```

✓ **Lectura.** Con la extensión **encendida** (el proyecto real) `/health` dice `"vdr":"enabled"` y `/base/ping` sigue respondiendo `pong`: **idéntico** al caso apagado, salvo la palabra `enabled/disabled`.

19.6 Falla cerrada: los valores no válidos no encienden

```
$ docker run -d --name vdr-off -e VDR_ENABLED=si vdr-ssi/vdr-service:local >/dev/null; sleep 5;
docker logs vdr-off 2>&1 | grep -E "VDR APAGADO|VDR HABILITADO" | sed "s/^[0-9:.\ ]*//"; docker rm -f
vdr-off >/dev/null
INFO vdr.main - VDR APAGADO (VDR_ENABLED=false): solo el camino base está activo
$ docker run -d --name vdr-off -e VDR_ENABLED=True vdr-ssi/vdr-service:local >/dev/null; sleep 5;
docker logs vdr-off 2>&1 | grep -E "VDR APAGADO|VDR HABILITADO" | sed "s/^[0-9:.\ ]*//"; docker rm -f
vdr-off >/dev/null
INFO vdr.main - VDR APAGADO (VDR_ENABLED=false): solo el camino base está activo
```

✓ **Lectura. Respuesta a P5:** `si` y `True` (y también `1`) **no** encienden: solo el texto exacto `true` lo hace. Es la *falla cerrada*: ante una configuración dudosa, lo riesgoso queda apagado.

🎯 Conclusión — evidencia del criterio 3

Verbo del criterio	Evidencia
El camino base opera	<code>/health</code> <code>UP</code> y <code>/base/ping</code> <code>pong</code> con la opción apagada (§19.2)
Sin el registro	Rutas del registro inexistentes (404), canal de escritura cerrado, 0 menciones de la base (§19.3 y §19.4)
Regresión	El camino base es idéntico encendido y apagado (§19.5)
Por defecto apagado	Arranca apagado sin variable; <code>si/True/1</code> no lo encienden (§19.1 y §19.6)
Prueba automática equivalente	<code>Erso004RegistryTest > criterio 3</code> (tres pruebas; no necesitan base de datos) (§20)

20. Pruebas automáticas y recorrido de extremo a extremo

Los laboratorios anteriores son **pruebas manuales** (las haces tú, viendo cada respuesta). Además existen **pruebas automáticas**: programas que repiten las mismas verificaciones en segundos y que se pueden ejecutar cada vez que cambia el código.

20.1 Ejecutar las pruebas de la ERSo 004

🔍 **Requisito.** Una base de datos desechable para las pruebas (`vdr-test-pg`, puerto 55432); **no toca el proyecto levantado**.

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect
$ docker ps --format '{{.Names}} {{.Ports}}' | grep vdr-test-pg
vdr-test-pg 127.0.0.1:55432->5432/tcp
$ TEST_DB_URL=jdbc:postgresql://127.0.0.1:55432/wallet_test ./gradlew :vdr-service:test --tests
'*Erso004*' --rerun-tasks -q 2>&1 | tail -3; echo "(sin salida = todas pasaron)"
(sin salida = todas pasaron)
$ python3 - <<'EOF'
> import re, glob, html
> f = glob.glob('vdr-service/build/test-results/test/*Erso004*.xml')[0]
```

```
> s = open(f, encoding='utf-8').read()
> tot = re.search(r'tests="(\d+)" skipped="(\d+)" failures="(\d+)"', s).groups()
> print(f"resultado: {tot[0]} pruebas, {tot[1]} omitidas, {tot[2]} fallos\n")
> for m in re.finditer(r'<testcase name="([^\"]+)"[^>]*?(>|>(.*)</testcase>)', s, re.S):
>     name = html.unescape(m.group(1)).removesuffix("(")
>     estado = "FALLA" if "<failure" in m.group(3) or "" else "PASA "
>     print(f" [{estado}] {name}")
> EOF
resultado: 7 pruebas, 0 omitidas, 0 fallos

[PASA ] criterio 2 - respaldo, perdida total y restauracion dejan el registro identico
[PASA ] criterio 1 - registro, actualizacion y trazabilidad son operativos
[PASA ] criterio 3 - con el registro encendido el camino base sigue igual
[PASA ] criterio 3 - con el registro apagado el camino base responde y los endpoints del VDR no
existen
[PASA ] solo el administrador puede respaldar y restaurar
[PASA ] criterio 3 - por defecto la extension esta apagada
[PASA ] paso 2 - el adaptador traduce DID a la forma del registro segun el metodo
```

Comando	Qué hace	Qué se busca
<code>docker ps ... grep vdr-test-pg</code>	Comprueba que la base desechable de pruebas existe	Tener dónde probar
<code>TEST_DB_URL=... ./gradlew :vdr-service:test --tests '*Erso004*' --rerun-tasks -q</code>	Ejecuta solo las pruebas de esta ERSO; TEST_DB_URL indica la base; --rerun-tasks fuerza a repetirlas	Que pasen
el script de Python	Lee el reporte y lista cada prueba con PASA / FALLA	Ver cuál prueba demuestra qué

✅ **Lectura.** 7 pruebas, 0 omitidas, 0 fallos. Cada prueba tiene el nombre del criterio que respalda:

Prueba	Respalda
criterio 1 - registro, actualizacion y trazabilidad son operativos	Criterio 1
criterio 2 - respaldo, perdida total y restauracion dejan el registro identico	Criterio 2
solo el administrador puede respaldar y restaurar	Criterio 2 (seguridad)
criterio 3 - por defecto la extension esta apagada	Criterio 3
criterio 3 - con el registro apagado el camino base responde y los endpoints del VDR no existen	Criterio 3
criterio 3 - con el registro encendido el camino base sigue igual	Criterio 3
paso 2 - el adaptador traduce DID a la forma del registro segun el metodo	Paso 2 / capacidad 2 (sin criterio propio)

Preguntas (con respuesta)

- ¿Cuántas pruebas dicen "criterio 3" y por qué más de una? **Tres**, porque el criterio tiene varias facetas: por defecto apagado, apagado responde y encendido no cambia nada.
- ¿Cuál no corresponde a ningún criterio? La del **paso 2** (el adaptador): el PDF pide ese paso pero ningún criterio lo comprueba, así que se añadió.
- Si se quitara del código la regla "solo el administrador", ¿qué prueba fallaría? solo el administrador puede respaldar y restaurar.

20.2 La demostración de que el criterio 3 no necesita base de datos

```
$ TEST_DB_URL= ./gradlew :vdr-service:test --tests '*Erso004*' --rerun-tasks -q 2>&1 | tail -3;
python3 - <<'EOF'
> import re, glob
> s = open(glob.glob('vdr-service/build/test-results/test/*Erso004*.xml')[0], encoding='utf-8').read()
> t, sk, f = re.search(r'tests="(\d+)" skipped="(\d+)" failures="(\d+)"', s).groups()
> print(f"SIN base de datos: {t} pruebas, {sk} omitidas (necesitan PostgreSQL), {f} fallos, {int(t)-int(sk)} ejecutadas")
> EOF
SIN base de datos: 7 pruebas, 4 omitidas (necesitan PostgreSQL), 0 fallos, 3 ejecutadas
```

✓ **Lectura.** Sin `TEST_DB_URL`, las pruebas que necesitan PostgreSQL se **omiten**, y las que no **se ejecutan y pasan**: son tres, las dos del camino base **apagado** (por defecto apagada y apagado responde) y la del adaptador. Eso prueba, además, que el camino base no depende de la base de datos.

20.3 El recorrido de extremo a extremo (extracto del registro)

El script `scripts/e2e.sh` recorre **todo el sistema real** (TLS, certificados, contenedores) y deja un registro. Este es el extracto de los bloques de esta ERSO, del último recorrido (`evidencias/e2e-20260929-155349.log`):

```
ERSo 2026-004 · paso 0 – el camino base y el VDR de extensión encendido

✓ PASS camino base responde (/base/ping)
✓ PASS health informa vdr=enabled
✓ PASS el puerto de la aplicación y de Postgres NO están publicados en el host

[... bloques de las ERSO 005 a 008 omitidos ...]

ERSo 2026-004 · respaldo/restauración y camino base con el registro apagado

HTTP 200 -> guardado en backup.json (30413 bytes)
✓ PASS C2: respaldo generado con checksum
Simulando pérdida total del estado del registro (TRUNCATE)...
✓ PASS tras la pérdida, el DID ya no se sirve (404)
HTTP 422
{
  "error": "CHECKSUM_MISMATCH",
  ✓ PASS un respaldo alterado se rechaza (CHECKSUM_MISMATCH)
HTTP 200
{
  "restored": {
    "entity_accounts": 4,
    "namespaces": 5,
    "did_documents": 4,
    "did_document_versions": 7,
    "operations": 7,
    "audit_log": 56
  },
  "documentsVerified": 4,
  "checksum": "sha256:558f3b19ca08fa27d7132d33a3536f2bb9f681a4a991630a3ba1dd1d96326d36"
}
✓ PASS restauración correcta con verificación de integridad
✓ PASS C2: tras restaurar, el DID de Avance se sirve idéntico (mismo hash)
✓ PASS C2: tras restaurar, el DID del laboratorio se sirve idéntico (mismo hash)
✓ PASS C2: tras restaurar, el DID del titular (Wallet Backend) se sirve idéntico (mismo hash)
```

```
✓ PASS C2: tras restaurar, el namespace comodín del Wallet Backend sigue siendo suyo
✓ PASS C2: el historial (4 versiones) sobrevive a la restauración
✓ PASS el DID desactivado sigue desactivado tras restaurar (410)

- C3: camino base con la opción APAGADA (VDR_ENABLED=false) -
✓ PASS C3: /base/ping responde con el registro apagado
✓ PASS C3: /health informa vdr=disabled
✓ PASS C3: los endpoints de did.json no existen (404)
✓ PASS C3: el canal de escritura ni siquiera está abierto (502 desde nginx)
- con el VDR apagado, la cartera sigue funcionando (el camino base no depende del registro) -
✓ PASS con el VDR apagado la cartera se activa igual (el camino base no depende del registro)
✓ PASS con el VDR apagado el DID del titular se crea y valida LOCALMENTE (no depende de un
registro obligatorio)
✓ PASS la publicación falla de forma controlada (502 VDR_ERROR), no con un error interno
✓ PASS la cartera sigue activa después del fallo de publicación
RESULTADO vdr-off: 4 PASS · 0 FAIL
Reactivando el registro...
✓ PASS el registro vuelve a estar encendido y conserva su estado
```

✓ **Lectura.** Cada línea ✓ PASS es una comprobación sobre el sistema real: el camino base responde, la aplicación y la base **no** están publicadas, el respaldo se genera con checksum, el respaldo alterado se rechaza, las huellas coinciden tras restaurar, y con `VDR_ENABLED=false` el camino base responde y el canal de escritura no está abierto.

Las cuatro últimas comprobaciones, sobre la cartera, pertenecen a otra ERSO (001-003): muestran que con el VDR apagado esos componentes **siguen funcionando**, lo que refuerza que la extensión está aislada.

PARTE V — PREGUNTAS Y RESPUESTAS

Esta parte reúne **todas las preguntas de comprensión** que se plantearon durante la capacitación sobre la ERSO 004, con su respuesta y el lugar de este informe donde se demuestra. Las predicciones de cada laboratorio están respondidas dentro de su sección; aquí se recopilan para consulta rápida.

21. Preguntas sobre los conceptos de base

#	Pregunta	Respuesta	Dónde se ve
1	Un DID tiene tres partes: <code>did</code> , <code>web</code> y <code>civica-desarrollo.avance.org.co:entidades:avance</code> . ¿Qué es cada una?	<code>did</code> es el esquema (dice "esto es un DID", como <code>http</code> en una dirección). <code>web</code> es el método (la receta que dice dónde viven los documentos). Lo demás es el identificador específico del método : aquí, el dominio más la ruta	§2.4, §15.4
2	¿Por qué un VDR "no es necesariamente una cadena de bloques"? Da un ejemplo del proyecto	Porque <i>VDR</i> es un rol (el lugar público donde se publican y consultan los documentos), no una tecnología. Aquí lo cumple un servicio web con PostgreSQL	§6, §7 (condición 2)
3	En TLS normal solo el servidor se identifica. ¿Qué agrega mTLS y por qué se usa solo en el canal de escritura ?	Agrega que el cliente también presenta un certificado , así se sabe qué entidad escribe. Se usa solo al escribir porque leer debe ser libre : cualquiera tiene que poder consultar una clave pública para verificar	§2.2, §15.5
4	Si roban el certificado cliente de una entidad, ¿qué otras cosas le faltarían al ladrón para cambiar su documento?	(a) El token (necesita el identificador y el secreto de la entidad) y (b) la firma del desafío con la clave privada vigente del DID . Además, el token solo vale para el espacio de esa entidad	§13.2, §17.6

#	Pregunta	Respuesta	Dónde se ve
5	Si un atacante lograra cambiar el <code>serviceEndpoint</code> del documento para que apunte a su servidor, ¿qué lograría y qué lo impide?	Podría suplantar al emisor : los clientes enviarían sus datos al servidor del atacante, que respondería con información falsa. Lo impide la defensa en capas : no puede escribir sin certificado, token y la firma con la clave privada del DID; además todo cambio queda en la auditoría	§13.2, §17.6
6	¿Por qué <code>hola y hola!</code> dan huellas totalmente distintas?	Por el efecto avalancha : cualquier cambio mínimo en la entrada cambia la huella por completo. Por eso sirve para detectar alteraciones	§2.1
7	¿Con qué clave se firma y con cuál se verifica?	Se firma con la privada y se verifica con la pública	§2.1
8	¿Firmar oculta el mensaje?	No . Firmar prueba autoría e integridad; el contenido sigue a la vista. Ocultar es <i>cifrar</i>	§2.1
9	¿Para qué sirve un nonce?	Para que una firma o petición capturada no se pueda reutilizar (impide el <i>replay</i>)	§2.3
10	La firma de un mismo mensaje hecha dos veces con la misma clave, ¿es idéntica?	No (incluye un valor aleatorio interno), pero ambas verifican . Por eso nunca se comparan firmas para saber si algo es igual: se <i>verifican</i> . Lo estable es el hash	Marco conceptual, laboratorio 1b
11	La clave pública siempre está a la vista. Entonces, ¿qué le faltó al atacante del experimento 3?	La clave privada vigente del DID. 💡 <i>La pública verifica; la privada firma</i> : la pública nunca es "lo que falta"	§17.6

22. Preguntas por lección y por criterio

Camino base y apagado por defecto (criterio 3)

Pregunta	Respuesta	Dónde
Con la extensión apagada, ¿qué dirá <code>/health</code> ?	<code>"vdr": "disabled"</code> (y el estado sigue en UP)	§19.2
¿Qué código dará el <code>did.json</code> y por qué?	404: la ruta no existe (no se registra cuando está apagada). No es "documento no encontrado"	§19.3
¿Se conecta a la base de datos?	No : no hay ni una línea de <code>hikari</code> , <code>postgres</code> o <code>jdbc</code> en su registro. La decisión está en <code>Main.kt</code> (<code>if (cfg.vdrEnabled)</code>)	§19.4

Pregunta	Respuesta	Dónde
¿Qué pasa si alguien escribe VDR_ENABLED=si ?	Queda apagado : solo true exacto enciende (<i>falla cerrada</i>)	§19.6
¿Por qué es mejor que true sin secreto no arranque?	Un error visible en el despliegue es barato; un servicio inseguro funcionando puede pasar desapercibido hasta que lo ataquen (<i>falla temprana</i>)	§16.3

Registro, actualización y trazabilidad (criterio 1)

Pregunta	Respuesta	Dónde
Para actualizar un DID en versión N, ¿qué If-Match se envía? ¿Y si se envía 0 ?	If-Match: N . Con 0 se obtiene 409 ALREADY_EXISTS (porque 0 significa "crear")	§17.3, §17.4
¿Con qué clave se firma una actualización y qué sería peligroso en el otro caso?	Con la vigente . Si bastara la nueva, cualquiera podría proponer un documento con su propia clave y adueñarse del DID	§17.4
Tras una actualización, ¿cuántas filas hay en did_documents y en did_document_versions para ese DID?	Una en did_documents (la foto de hoy) y una por versión en did_document_versions (el álbum)	§17.2, §17.8
¿Qué diferencia hay entre did_documents y did_document_versions , y por qué las dos?	La primera es el estado actual (rápido de consultar); la segunda, el historial completo. Perder el historial dejaría sin trazabilidad; leer "la última versión" del historial en cada consulta sería más lento	§12.8
¿Por qué la confirmación lee la URL pública en vez de fiarse de lo que acaba de escribir?	Porque escribir en la base no garantiza lo que el mundo ve : nginx, una caché o una ruta mal escrita pueden servir otra cosa. El servicio hace lo que haría un cliente	§13.1, §17.4
Si la confirmación leyera directo de la base de datos, ¿detectaría un nginx roto?	No . Solo leería lo que acaba de escribir; el fallo está en el camino público, que no vería	§13.1
¿Por qué el historial está protegido en la base y no solo en el código?	Porque así sigue protegido ante un error de programación o ante alguien con acceso directo a la base	§17.9
¿Qué evita If-Match ? ¿Y Idempotency-Key ?	If-Match evita que dos escrituras simultáneas se pisen (la segunda recibe 412). Idempotency-Key evita que un reintento duplique versiones	§2.3, §17.7

Respaldo y recuperación (criterio 2)

Pregunta	Respuesta	Dónde
Restaurar sobre un registro con datos, ¿se restaura?	Se rechaza (409 REGISTRY_NOT_EMPTY): evita mezclar historiales y pisar datos vivos	§18.4
Tras vaciar las tablas, ¿qué código da el did.json y sigue vivo el servicio?	404 ; el servicio sigue vivo, lo que desapareció fueron los datos	§18.5

Pregunta	Respuesta	Dónde
Si se altera un hash del respaldo, ¿qué dice el servidor y cuántos DID quedan?	422 CHECKSUM_MISMATCH y 0 DID: nada queda a medias	§18.6
¿Qué concepto explica que no quede nada a medias?	La transacción (todo o nada) más la validación del checksum antes de escribir	§18.6
Tras restaurar el bueno, ¿las huellas son iguales? ¿Y el DID desactivado?	Iguales ; el desactivado sigue en 410	§18.8
¿Por qué el respaldo incluye audit_log y el historial?	Porque el criterio 1 exige trazabilidad ; sin ellos el registro funcionaría pero no podría probar quién hizo qué	§18.3, §18.8
Si el respaldo no incluyera la auditoría, ¿el DID seguiría funcionando? ¿Qué se perdería?	Sí funcionaría (el documento público se sirve desde las tablas de documentos y versiones), pero se perdería la trazabilidad : quién hizo qué, los intentos rechazados y la evidencia de publicación	§18.8
¿Qué dos cosas constituyen "la verificación" del respaldo?	(1) El checksum , que rechaza respaldos alterados; (2) la comparación posterior de huellas, conteos y documentsVerified	§18.6–§18.8

Capacidades y condiciones

Pregunta	Respuesta	Dónde
¿Qué contenedores publican puertos hacia el exterior?	Solo nginx	§15.1
¿Funciona entrar directo a la aplicación saltándose nginx? ¿Y desde dentro de la red de Docker?	Desde fuera no (código 7); desde dentro sí . Son ciertas a la vez porque depende de dónde preguntes. <i>Aislado no es inexistente</i>	§15.2
Un PUT por la puerta de lectura, ¿qué da? ¿Y /admin/v1/... por ahí?	403 y 404 respectivamente	§15.5
Si se detiene el VDR, ¿qué pasa con los demás servicios?	Siguen igual; el did.json da 502 desde nginx. El fallo queda contenido	§16.1
El 502 lo da nginx, no la aplicación. ¿Por qué es la señal correcta?	Porque prueba que la puerta de entrada y los demás servicios siguen vivos: el fallo no se propagó	§16.1
¿Basta con escribir otro adaptador para soportar otro método DID?	No del todo : hay que escribirlo y generalizar el tipo DidWebId que hoy devuelve la interfaz	§16.2
¿Cuántas pruebas dicen "criterio 3" y por qué más de una?	Tres : por defecto apagado; apagado responde; encendido no cambia nada. El criterio tiene varias facetas	§20.1
¿Cuál prueba no corresponde a ningún criterio?	La del paso 2 (adaptador): el PDF pide ese paso, pero ningún criterio lo comprueba	§20.1

PARTE VI — CÓMO RESPONDER A CADA CRITERIO

Un criterio de aceptación se responde con tres elementos: **afirmación + prueba + dónde verla**. A continuación, la redacción modelo.

23. Redacción modelo de las tres respuestas

Criterio 1 — Registro y actualización operativos

Afirmación. El registro y la actualización de entradas del VDR son operativos y quedan trazados: un documento se crea una sola vez, se actualiza únicamente con la versión esperada y la firma de la clave vigente, y cada operación —incluidos los intentos rechazados— queda en un historial que no puede borrarse ni modificarse.

Prueba. (1) Actualización legítima → 200 CONFIRMED, versión N+1, y el hash del documento público es idéntico al hash de la versión registrada. (2) Volver a crear el DID → 409 ALREADY_EXISTS. (3) Escritura firmada con una clave ajena → 403 INVALID_PROOF. (4) Escritura con versión desactualizada → 412 VERSION_CONFLICT. (5) DELETE y UPDATE sobre el historial → tabla append-only: ... no permitido.

Dónde verla. Laboratorio del criterio 1 (§17; scripts/lab-004-registro.sh); prueba automática Erso004RegistryTest > criterio 1 - registro, actualizacion y trazabilidad son operativos (§20); registro de auditoría con WRITE_UPDATE, PUBLICATION_CONFIRMED, PROOF_REJECTED y VERSION_CONFLICT.

Criterio 2 — Respaldo y recuperación verificados

Afirmación. El estado completo del registro (cuentas, espacios de nombres, documentos, historial, operaciones y auditoría) puede respaldarse y recuperarse tras una pérdida total, y la recuperación se verifica: el registro restaurado es idéntico al original.

Prueba. (1) Respaldo → 200, formato vdr-backup/1 con checksum y seis tablas. (2) Restaurar sobre datos vivos → 409 REGISTRY_NOT_EMPTY. (3) Tras vaciar las tablas, los documentos dan 404. (4) Respaldo con un hash alterado → 422 CHECKSUM_MISMATCH y 0 documentos restaurados. (5) Respaldo íntegro → 200 con documentsVerified igual al número de DID. (6) Las huellas públicas, los conteos y el estado del DID desactivado (410) **coinciden** con los de antes.

Dónde verla. Laboratorio del criterio 2 (§18; scripts/lab-004-respaldo.sh); pruebas automáticas criterio 2 - respaldo, perdida total y restauracion dejan el registro identico y solo el administrador puede respaldar y restaurar (§20); extracto del E2E (§20.3).

Criterio 3 — Camino base sin el registro

Afirmación. Con la opción apagada —que es el valor por defecto— el camino base opera sin cambios, las rutas del registro no existen, el canal de escritura no se abre y no se accede a la base de datos.

Prueba. (1) Sin definir `VDR_ENABLED`, el servicio arranca sin base de datos y registra "VDR APAGADO". (2) `/health` → UP con `vdr: disabled`; `/base/ping` → pong. (3) `did.json` → 404 (ruta inexistente). (4) El puerto de escritura no escucha. (5) El registro del servicio tiene **0** menciones de la base de datos. (6) Encendido, el camino base responde igual. (7) `si`, `True` y `1` no encienden la extensión.

Dónde verla. Laboratorio del criterio 3 (§19; `scripts/lab-004-camino-base.sh`); tres pruebas automáticas `criterio 3 ...` que no requieren base de datos (§20.2); extracto del E2E (§20.3).

24. Lista de comprobación para quien acepta

✓	Criterio	Qué ejecutar	Resultado esperado
☐	1 Registro y actualización	<code>bash scripts/lab-004-registro.sh</code>	Experimento 1 409; 2 200 CONFIRMED; hashes iguales; 3 403 INVALID_PROOF; 4 412 VERSION_CONFLICT; historial y auditoría; append-only en DELETE y UPDATE
☐	2 Respaldo y recuperación	<code>bash scripts/lab-004-respaldo.sh</code>	409 · 404 · 422 CHECKSUM_MISMATCH y 0 documentos · 200 documentsVerified · huellas iguales antes y después
☐	3 Camino base	<code>bash scripts/lab-004-camino-base.sh</code>	disabled / pong · 404 · puerto 8081 cerrado · 0 menciones de la base · encendido igual · <code>si</code> / <code>True</code> / <code>1</code> apagados
☐	Capacidades y aislamiento (pasos 1-2)	<code>bash scripts/lab-004-capacidades.sh</code>	Solo nginx publica; exterior sin acceso directo; adaptador; dos puertas
☐	Condiciones	<code>bash scripts/lab-004-condiciones.sh</code>	502 con VDR detenido y los demás vivos; <code>invalidDid</code> ; <code>true</code> sin secreto no arranca
☐	Pruebas automáticas	<code>bash scripts/lab-004-pruebas.sh</code>	7 pruebas, 0 omitidas, 0 fallos
☐	Recorrido E2E	<code>bash scripts/e2e.sh</code> (≈5 min; reinicia el entorno)	RESULTADO: ... PASS · 0 FAIL

PARTE VII — LÍMITES, RIESGOS Y OPERACIÓN

25. Lo que este informe NO demuestra (dilo tú primero)

Declarar los límites es parte de la evidencia: protege la credibilidad de todo lo demás.

Límite	Explicación
TLS de laboratorio	Los certificados los firma una autoridad propia . Para producción hace falta una autoridad reconocida (por ejemplo, Let's Encrypt) en el canal de lectura y una CA privada para las entidades
No se probó en el dominio real ni desde internet	Todo se ejecutó en una máquina local usando <code>--resolve</code>
Solo existe <code>did:web</code>	El adaptador está pensado para otros métodos, pero la interfaz aún devuelve un tipo con forma de <code>did:web</code> (<code>DidWebId</code>)
El respaldo contiene datos sensibles	No incluye secretos de clientes, pero sí el historial completo y la auditoría . Debe guardarse cifrado y con acceso restringido
La restauración solo entra en un registro vacío	Es una decisión de seguridad; implica que restaurar sobre un registro dañado exige vaciarlo antes
No hay limitación de tasa ni rotación automática de secretos	Fuera del alcance de esta ERSO
Confianza en cabeceras de mTLS	La aplicación confía en que nginx le informa la identidad del certificado. Es seguro solo porque el puerto de la aplicación no se publica (probado en §15.1) y nginx las sobrescribe siempre
Implementación propia de Base58/Multikey	Verificada por ida y vuelta y por el prefijo <code>zDn</code> , no contrastada con vectores de terceros
Fechas	La fecha de desarrollo (21-sep) y de prueba (23-sep) de la ERSO ya habían pasado cuando se hizo esta implementación
Firmas y plantilla de pruebas	Las firmas de la tabla de actividades y la plantilla de pruebas funcionales corresponden a las personas responsables; este informe aporta evidencias técnicas
Los criterios no cubren los pasos 1 y 2	Se cubrieron con evidencia propia (§15 y §16), pero no son parte formal de la aceptación

26. Si algo no sale como en el informe

Síntoma	Causa más probable	Qué hacer
<code>curl: (60) SSL certificate problem</code> o <code>HTTP 000</code> con código de salida 60	<code>curl</code> no conoce nuestra CA	Añade <code>--cacert certs/ca.crt</code> (está en <code>\$PUB</code>)

Síntoma	Causa más probable	Qué hacer
HTTP 000 en un <code>curl</code> al puerto 8444	Usaste las opciones del puerto 8443 (<code>\$PUB</code>) en vez de las del 8444 (<code>\$CRED</code>), o falta un espacio al pegar	Usa <code>\$CRED</code> ; revisa el comando
<code>código de salida 7</code>	No se pudo conectar	Revisa <code>docker compose ps</code> y que el proyecto esté levantado
Variables vacías (<code>\$D</code> , <code>\$PUB</code> ...)	Abriste un terminal nuevo	Repite §12.4 y §12.6
Al pegar comandos largos se corrompen caracteres	La terminal pierde texto al pegar	Ejecuta el script equivalente (<code>bash scripts/lab-004-*.sh</code>)
HTTP 404 en el <code>did.json</code> de Avance	No existen los datos de ejemplo	Ejecuta <code>bash scripts/e2e.sh</code>
<code>credential</code> se reinicia sin parar	Falta <code>evidencias/work/avance.json</code>	Lo crea el recorrido E2E
<code>permission denied</code> al ejecutar un script	Falta permiso de ejecución	<code>bash scripts/archivo.sh</code> (con <code>bash</code> delante)
<code>REGISTRY_NOT_EMPTY</code> al restaurar	El registro tiene datos	Es lo esperado; vacíalo primero (laboratorio del criterio 2, paso 4)
Las huellas cambiaron después de restaurar	No debería ocurrir	Compara el checksum del respaldo y revisa que no usaste un respaldo antiguo

27. Operación diaria

Quiero...	Comando (desde <code>deploy/</code>)	Nota
Levantar el proyecto	<code>docker compose up -d</code>	
Ver el estado	<code>docker compose ps</code>	
Ver los registros de un servicio	<code>docker compose logs --tail=50 vdr</code>	
Detenerlo conservando los datos	<code>docker compose down</code>	Los datos viven en un volumen aparte
Reiniciarlo	<code>docker compose down && docker compose up -d</code>	
Detener solo el VDR	<code>docker compose stop vdr</code>	Los demás siguen
 Borrar también los datos	<code>docker compose down -v</code>	Destruyitivo: elimina el volumen de PostgreSQL
Reconstruir tras cambiar código	<code>cd .. && ./gradlew installDist && cd deploy && docker compose --profile tools build</code>	

ANEXOS

Anexo A — Los scripts de laboratorio

Script	Qué hace	¿Modifica datos?
scripts/gen-dev-certs.sh	Crea CA y certificados de laboratorio (idempotente)	Crea deploy/certs
scripts/gen-env.sh	Crea o actualiza deploy/.env	Crea deploy/.env
scripts/lab-004-capacidades.sh	Laboratorio A (§15)	No
scripts/lab-004-condiciones.sh	Laboratorio B (§16)	Detiene el VDR unos segundos
scripts/lab-004-registro.sh	Criterio 1 (§17)	Agrega una versión al DID del laboratorio
scripts/lab-004-respaldo.sh	Criterio 2 (§18)	Vacía y restaura (queda idéntico)
scripts/lab-004-camino-base.sh	Criterio 3 (§19)	No (copia temporal)
scripts/lab-004-pruebas.sh	Pruebas automáticas (§20)	No (base desechable)
scripts/e2e.sh	Recorrido completo	 Reinicia el entorno

Anexo B — Autocomprobación del marco conceptual (con respuestas)

#	Pregunta	Respuesta
1	¿Por qué hola y hola! dan huellas distintas?	Efecto avalancha: un cambio mínimo cambia toda la huella
2	¿Con qué clave firmas y con cuál verificas?	Firmas con la privada; verificas con la pública
3	¿Firmar oculta el mensaje?	No; solo prueba autoría e integridad
4	¿Para qué sirve un nonce?	Para que una firma o petición capturada no se reutilice
5	¿Qué tres cosas da TLS?	Cifrado, integridad y autenticación del servidor
6	¿Quién firmó server.crt y qué pasa con otra CA?	La CA de laboratorio; con otra, la validación falla porque no encuentra al emisor
7	¿Diferencia entre TLS y mTLS?	En mTLS también el cliente presenta certificado
8	¿Por qué la aplicación no publica su puerto?	Porque confía en cabeceras que pone nginx; si el puerto estuviera publicado, alguien podría inventarlas

#	Pregunta	Respuesta
9	¿Diferencia entre 401 y 403?	401: no sabemos quién eres. 403: sabemos quién eres pero no tienes permiso
10	¿Por qué el token solo no basta para escribir?	Prueba que conoces un secreto, no que controlas la clave del DID
11	¿Qué evita If-Match ?	Que dos escrituras concurrentes se pisen (412)
12	¿Qué evita Idempotency-Key ?	Que un reintento duplique versiones
13	¿Qué contiene un DID Document y qué no debe contener?	Claves públicas, para qué sirve cada una, controlador y servicios; jamás claves privadas ni datos civiles
14	¿Cómo se convierte un <code>did:web</code> en URL?	Los <code>:</code> tras el dominio pasan a <code>/</code> y se añade <code>did.json</code>
15	¿Resolver frente a dereferenciar?	Resolver: DID → documento. Dereferenciar: DID URL (con <code>#</code>) → una parte concreta
16	¿Por qué el VDR es un rol?	Describe una función que puede cumplir una cadena de bloques, un servidor web u otra cosa
17	¿Qué garantiza una transacción?	Que los cambios se apliquen todos o ninguno
18	¿Diferencia entre <code>did_documents</code> y <code>did_document_versions</code> ?	Estado actual / historial
19	¿Para qué el checksum del respaldo?	Detectar que fue alterado antes de restaurarlo
20	¿Por qué restaurar solo sobre un registro vacío?	Para no mezclar historiales ni pisar datos vivos

Anexo C — Ejercicios del método (con respuesta modelo)

Ejercicio 1. *Aplicar la receta al criterio 4 de la ERSO 005 ("un `id` desajustado se rechaza").* | Campo | Respuesta | `---` | Verbo / objeto / evidencia | *Se rechaza* / un documento con `id` que no corresponde al DID / pruebas negativas | | Afirmación comprobable | Un documento cuyo `id` no coincide con el DID de su ruta no se publica y no se sirve | | Casos negativos | Documento con `id` de otra entidad; DID de ciudadano sin espacio reservado | | Mecanismo | El validador compara `id` con el DID esperado (`ID_MISMATCH`, 422); el espacio no reservado da 412 | | Prueba automática | `Erso005DidWebTest > criterio 4 ...` |

Ejercicio 2. *Dos casos negativos adicionales para la escritura autenticada (ERSO 006).* (a) Un token **vencido** (diez minutos) → 401. (b) Un certificado cliente válido pero **de otra entidad** con las credenciales de la primera → 403 `MTLS_REQUIRED`. *Capa que los frena:* el token y el vínculo certificado↔entidad.

Ejercicio 3. *¿Por qué la protección `append-only` debe estar en la base de datos y no en el código (ERSO 008, criterio 7)?* Porque un *trigger* funciona **aunque el código tenga un error o alguien acceda directo a la base**; una regla solo en el código se pierde si el código falla o se evita.

Anexo D — Referencias

- W3C — *Decentralized Identifiers (DIDs) v1.1* y *DID Resolution v1*.
- W3C CCG — *did:web Method Specification*.

- W3C — *Verifiable Credentials Data Model* (concepto de *Verifiable Data Registry*).
 - IETF — RFC 7515 (JWS), RFC 6749 (OAuth 2.0), RFC 8705 (mTLS), RFC 9110 (*If-Match* , *ETag*).
 - DIF — *DID Registration* (patrón de API de gestión).
 - Enlaces y estado de cada especificación: `docs/VERSIONES-NORMATIVAS.md` (**confirmar siempre la versión vigente**).
-

Fin del informe. Elaborado para la ERSO 2026-004 del proyecto Identidad Digital Soberana SSI. Las salidas de terminal se obtuvieron ejecutando los comandos reales el 1 de octubre de 2026; los valores variables (identificadores, fechas, huellas) cambian en cada ejecución, no así los patrones que se explican en cada sección.